claude-windows / e83b09d8-e113-4d4d-9411-cd80ae217af9

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9.jsonl`
- SHA-256: `64b5d234255ec410c85ddc70bbab487413e31ffb644f606ab19ea2c4f916973b`
- Source modified: `2026-07-07T17:39:25+00:00`
- Imported at: `2026-07-08T15:59:16+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `e83b09d8-e113-4d4d-9411-cd80ae217af9`

Transcript

1. user (2026-07-07T15:02:04.792Z)

In the cloud-serving dispatch path, `CloudRunCompute.run_infer` (backend/compute/cloud_run.py, ~line 213) waits for job completion by BUCKET-POLLING the done-marker only ? it never consults the Cloud Run execution status. If the worker container crashes before writing the marker (e.g. an old worker image argparse-aborts on an unknown flag like `--skip-validation`, an OOM, or any pre-marker exception), the dispatcher polls the full `deployment.remote_poll_max_wait_sec` (~65 min default, config/models.py) and then fails opaquely as a generic `cloud dispatch error` (rc 4), with the real cause visible only in the Job execution log.

Harden `run_infer` so it also polls the Cloud Run Executions API for a terminal Failed/Cancelled state (the client already has `run_job`/`cancel_execution` plumbing) and fails fast + diagnostically when the execution has terminated without a marker ? collapsing the ~65-min opaque hang to a prompt, actionable failure for the whole class of "worker crashed before the done-marker" cases. Keep the marker as the success signal; add the execution-status check as a fast negative path. Add a test with a fake client whose execution reports Failed while no marker is written, asserting a fast non-4-or-clearly-labelled failure.

Repo: khelsutra-guru/badminton-highlight-indexer (real code under C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer). Branch from origin/master; pre-merge gate = full suite + ruff + mypy + coverage floor 70. Surfaced by the adversarial review of PR fix/worker-skip-revalidation-on-dispatch (worker skip-revalidation), which added a deploy-together note as the interim mitigation.

2. assistant (2026-07-07T15:02:17.426Z)

I'll start by getting oriented ? my memory says the real code is under `khelsutra-guru/badminton-highlight-indexer`, and I need to pull before reading per convention. Let me do initial recon in parallel.

3. user (2026-07-07T15:02:19.517Z)

```
On branch perf/validation-shared-decode-pass
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        score_golden_gate.py
        score_local.py

nothing added to commit but untracked files present (use "git add" to track)
---BRANCH---
perf/validation-shared-decode-pass
---REMOTE---
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
```

4. user (2026-07-07T15:02:19.589Z)

```
1       ---
2       name: session-handoff-gpu-resident
3       description: "Start here to pick up the GPU-resident WASB rollout (issue #506) ?
implementation PR #507 is open; golden gate next"
4       metadata:
5         type: project
6       ---
7
8       # Session Handoff ? GPU-resident WASB inference (rollout)  ·  issue #506 · PR #507
MERGED
9
10      ## SHIPPED 2026-07-07 ? GPU-resident NVDEC decode is LIVE in cloud-serving
11      Full arc done: PR #507 (feature) + PR #508 (cloud-serving preset default =
nvdec_resident) both merged to master (44a2f47, 143dbb9). The `rally-worker` Cloud Run Job
(asia-southeast1) runs the new image `rally-worker:latest` `sha256:465a9250?` (torch 2.6 +
PyNvVideoCodec 2.1.0 + baked NVDEC libs) with `WASB_DECODE_BACKEND=nvdec_resident` set ? **nvdec
is the live serving default**, validated on the real L4 (GX010128 served trajectory reproduced
the VM: 26479?26481 visible; cpu+nvdec+canary smokes all ?). Record:
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md`.
12      - **ROLLBACK levers:** `gcloud run jobs update rally-worker --region asia-southeast1
--remove-env-vars WASB_DECODE_BACKEND` (? torch-2.6 cpu path, same image); or `--image ?/rally-
worker@sha256:3bfefaf6?` (? prior torch-1.11 image).
13      - **PRs:** #507 (feature) + #508 (preset default) + #509 (dispatch forwarding + release
hardening) all merged. master @ 66756ad.
14      - **DONE ? control-plane + worker both on image D2 `sha256:09d2edf7` (built from master
66756ad = #507+#508+#509).** Redeployed 2026-07-07: `gcloud run services update rally-control-
plane --image ?:latest` ? rev **00017-qbd** (100% traffic, booted healthy: uvicorn up, cloud-
serving profile ACTIVE, edge-auth on); `gcloud run jobs update rally-worker --image ?:latest`.
So the #509 dispatch **forwarding** + #508 preset default are now LIVE in the control-plane
image. NB the FIRST control-plane deploy this session used the stale `465a9250` (built from
44a2f47, pre-#508/#509) ? rev 00016; superseded by 00017. Rollback: `gcloud run services update-
traffic rally-control-plane --to-revisions rally-control-plane-00015-kp9=100` (last pre-nvdec
rev) or `--to-revisions ?-00016-?` .
15      - **`WASB_DECODE_BACKEND=nvdec_resident` env is still set on the worker Job** ? now
REDUNDANT with the forwarded --set + preset (both say nvdec, env just wins), kept as belt-and-
suspenders + manual override. Safe to remove ONLY after a real /api/process dispatch confirms
the forwarding path live (needs a CF Access JWT ? not verifiable this session; forwarding is
deployed + unit-tested but not live-dispatch-verified).
16      - `DEPLOY_REPORT_?gpu-resident-nvdec_2026-07-07.md` is current: PR #510 added the §5
"Update" (D2 redeploy of control-plane rev 00017 + worker) and corrected §6. All four PRs
merged: #507 #508 #509 #510; master @ 3be0935.
17      - **Wrinkles fixed (#509):** dispatch now forwards
indexing.wasb.{decode_backend,batch_size,prefetch_batches} (worker has no DEPLOYMENT_PROFILE ?
never applied the preset; batch tuning was silently lost too); added `.gcloudignore` (Cloud
Build had none ? relied on the .gitignore fallback to skip 8 GB output/scratch); added
`deploy/cloudrun/post_deploy_smoke.ps1`. See [[gpu-vm-setup-gotchas]].
18      - WASB-C3 weights-license gate still open (unchanged; pre-existing).
19
20      ## Live CUJ 2026-07-07 ? release PROVEN, 3 pre-existing gaps surfaced (all filed as
spawned tasks)
21      Owner ran a real /api/process CUJ ("Video 1 - Badminton.MP4", a 5312x2988 **10-bit
HEVC** 120Mbps clip). Outcomes:
22      - **#509 forwarding PROVEN live**: the cloud dispatch put `--set
indexing.wasb.decode_backend=nvdec_resident --set ?batch_size=64 --set ?prefetch_batches=4` into
the worker exec args. And a sharp-clip green run on the D2 worker (exec `rally-worker-ql56s`,
env-driven nvdec) = 19 rallies, success. Serving path is GREEN for valid footage.
23      - **Gap 1 ? slow validation (task_dcee8bf0):** ~125 random `cap.set(POS_FRAMES)` seeks
across scene_cut(100)/blur(15)/relevance(10), each their own VideoCapture, on 5.3K 10-bit HEVC
on the CPU-only control-plane ? ~6.5 min. Fix: single sequential/keyframe pass.
24      - **Gap 2 ? compute=local footgun (task_81572190):** SPA sent compute=local ? in-process
on the weightless control-plane ? "weights not found" fail. Guard/redirect it.
25      - **Gap 3 ? worker re-validates with DEFAULT thresholds (task_72d98f83):** control-plane
```

min_blur=8.0 (baked) NOT forwarded; worker re-validates at default 20.0 and REJECTED this soft clip (sharpness 11.9) ? rc2. This is what failed the CUJ on the cloud path. Fix: skip worker re-validation on dispatch (control-plane is authoritative) or forward validation config.
26   - **Validation-latency design (my rec):** don't move validation to GPU standalone. (1) stop the double-validation, (2) kill the seeks (sequential/keyframe), (3) end-state = ffprobe metadata pre-gate on control-plane + fold content checks (blur/court) INTO the worker's existing GPU decode pass (decode once, validate+detect together). All 3 gaps are pre-existing, NOT caused by the nvdec release.
27
28   ## (historical) release/rollout phase ? superseded by SHIPPED above
29   PR #507 squash-merged to master as **44a2f47** (branch deleted). Pre-merge gate reproduced locally: full suite 1764 passed / coverage 85.02% / ruff+mypy clean; CI 11/11 green; 2 optional review follow-ups landed (PyNvVideoCodec==2.1.0 pin in both deploy files; decode_backend persisted in native-runner detection_params). Real-L4 validation + golden gate PASS + latency/resource A/B all done (see PR #507 comment + below). **Remaining = the serving release (image rebuild + gated nvdec flip); decode_backend still defaults cpu so nothing in prod has changed yet.**
30
31   ### Serving release plan (live infra: Job `rally-worker` + service `rally-control-plane`, asia-southeast1; ONE image `rally-worker:latest` drives both ? gen_service_yaml.py:193)
32   - **Phase A (ship image, no behavior change):** `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project gen-lang-client-0306026826 .` ? new `rally-worker:latest` (torch2.6+pynvc2.1.0+NVDEC libs). Push does NOT change prod (Job/service pin a digest). Cut over via `gcloud run jobs update rally-worker --region asia-southeast1 --image ?:latest` + redeploy service (`gen_service_yaml.py` ? `services replace`). decode_backend defaults cpu ? cpu path under torch2.6 (validated F1 0.576?0.582). Smoke 1 clip. Rollback: `jobs update --image <old digest>`.
33   - **Phase B (enable nvdec, gated + reversible):** flip via WORKER env `WASB_DECODE_BACKEND=nvdec_resident` (native runner reads env first ? no code/preset change, instant rollback by removing it). ? THE untested risk: NVDEC on the real Cloud Run L4 ? the image bakes libnvcuvid 580.159.03; Cloud Run's host-driver ABI is unverified there (only tested on a self-managed L4 VM). MUST smoke 1 clip on the Job before trusting it. Then canary; then durable default = add `decode_backend: nvdec_resident` to the cloud-serving preset (presets.py) + rebuild + Launch Report.
34   - Cost: build ~$0.3; each Job smoke ~$0.4?0.7 (L4). Rollback levers at every step.
35
36
37   ## You are here
38   The implementation is DONE and **PR #507 is open** (branch `feat/506-gpu-resident-nvdec`, 2026-07-07): default-OFF `indexing.wasb.decode_backend = "cpu" | "nvdec_resident"`, the validated NVDEC?affine-`grid_sample`?GPU-normalize path wired into `backend/pipeline/detectors/wasb_infer.py` (`NvdecResidentFrontend` + `run_video_nvdec_resident`, same resumable cache keyed by decode_backend), native-runner threading + healthcheck gates (CUDA + PyNvVideoCodec), and the env bump (wasb env ? py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec; libnvcuvid/libnvidia-encode staged in `deploy/cloudrun/Dockerfile` + `deploy/digitalocean/gpu_setup.sh`). Verified locally: 1764 tests green, coverage 85% (floor 70), ruff/mypy/link/leak clean; a 14-agent adversarial review confirmed 7 minor findings, all fixed (notably `padding_mode="zeros"` = cv2 BORDER_CONSTANT for non-16:9 sources). Research evidence unchanged: [[gpu-resident-modernization]], issue #506 comments (F1 0.574 ? baseline 0.582 on GX010128 at SERVED, ~1.7×, GPU 47?79%).
39
40   ## Real-L4 validation ? DONE + golden gate PASS 2026-07-07 (this session, ~$3 of the $100 budget; VM torn down)
41   Validated end-to-end on an L4 (`rally-506`, torn down clean, no orphan disks) ? full evidence in the PR #507 comment. All GREEN: (1) `gpu_setup.sh` builds the env clean (py3.10 + torch 2.6.0+cu124 cuda True + PyNvVideoCodec 2.1.0 + NVDEC lib extraction); (2) on-box **parity gate PASS** (`pytest -k parity`, 480s ? needed test-name fix 9267255, the documented `-k parity` selected 0 tests); (3) crash/resume byte-identical; (4) cpu?nvdec cache isolation correct; (5) **Cloud Run image builds + `--gpus all` smoke PASS** (baked NVDEC libs decode; native healthcheck ok with nvdec_resident); throughput 43.7 fps / GPU 82% on GX010128.
42   **Golden gate @ SERVED = PASS** (10 baseline-comparable clips, 445 rallies): mean ?f1@0.5 **+0.020**, mean ?tolF1 +0.006, worst ?0.009 (**GX010128 must-pass: 0.574 vs 0.582 ?**), 8/10 improved-or-flat. The other 5 golden clips are 4K-sourced (baselines are 1080p) so a vs-baseline number would conflate resolution with decode ? the parity gate already covers nvdec?cpu on identical pixels, so they add nothing to the verdict.

```
43      **Latency + resource A/B** (2nd L4 `rally-506b`, torn down; GX010128, batch 8, telemetry
in `gs://?nightly/dev/506/results/bench/`): nvdec **43.7 fps @ GPU 83% / CPU 30%** vs cpu-
stream(no-prefetch) 15.0 fps @ GPU 28% / CPU 49%; GPU mem +170 MiB (of 24GB), RAM ~2?3 GB flat.
The win is the **bottleneck flip (GPU?, CPU?)**; the 2.9× is vs the untuned streaming path ? vs
prefetch-tuned cpu the realistic end-to-end is ~1.6?1.8× (#506 spike). All in the PR #507
comment.
44
45      ## Staged assets (durable)
46      - **Golden video working set**: all 14 originals in `gs://khelsutra-rally-
corpus/videos/golden/<name>.mp4` + the 4 baseline-source 1080p proxies in `videos/golden_1080p/`
(mahadevpura_2, Badminton_BXH_2, Boxhill_Doubles, GX020094). Frame-count-verified against each
golden trajectory. (GOLDEN_VIDEOS.md could record this GCS location in a follow-up ? kept out of
PR #507 to not muddy its scope.)
47      - Gate trajectories: `gs://khelsutra-rally-nightly/dev/506/results/golden/*_nvdec.csv`.
Re-score with repo-root `score_golden_gate.py` (untracked dev tool, like `score_local.py`;
labels from `?corpus/labels/`, baselines from `?corpus/trajectories/`).
48
49      ## Next steps
50      1. **Merge PR #507** ? real-L4-validated + golden-gate PASS; CI green locally.
51      2. **Flip the default**: owner Launch Report ? set `decode_backend=nvdec_resident` in
the cloud-serving preset (`backend/config/presets.py`). The gate that gated this is now green.
52
53      ## Ramp-up kit
54      - PR #507 (full change description + verification record) and issue #506 (all comments ?
measured evidence).
55      - New tests: `tests/test_wasb_nvdec_resident.py` (grid/cv2 parity incl. non-16:9, cache
keying, CLI guards, the skippable on-box parity gate).
56      - Reference code still in `scratch/gpu_resident/` (`exp_t2_full.py`, `score_local.py`,
`t2setup_env.sh`, `telemetry.sh`).
57
58      ## Key decisions (don't relitigate)
59      - GPU resize MUST be affine `grid_sample` matching `cv2.warpAffine` ? NOT
`F.interpolate` (F1 0.58?0.07; only shows on hard clips). Locked by unit tests now.
60      - `padding_mode="zeros"` (cv2's BORDER_CONSTANT(0)), not "border" ? matters only for
non-16:9 sources; identical to the validated run on 16:9.
61      - PyNvVideoCodec (MIT) for GPU decode; TorchCodec couldn't load on the DL image.
62      - Score with SERVED preset (cap6+R4); raw (0.02,2.0,2.0) is degenerate.
63      - Default-OFF; nvdec/cpu detection caches never cross-replay (manifest keyed by
decode_backend).
64      - WASB weights/NYCU-dataset license (WASB-C3) is a separate launch gate, not a perf
blocker.
65
```

5. user (2026-07-07T15:02:20.107Z)

```
1       ---
2       name: actual-repo-location
3       description: "The badminton-highlight-indexer code lives under khelsutra-guru/, not the
primary working dir stub"
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: df5d5fb2-534a-447a-8181-b222f94655f3
8       ---
9
10      The primary working directory `C:\Users\avidu\Projects\badminton-highlight-indexer` is a
near-empty
11      **stub** ? it contains only `AGENTS.md`, an empty `.claude/`, and
`output/nightly_regression/`
12      (a `LATEST_STATUS.txt`). The actual `khelsutra-guru/badminton-highlight-indexer`
codebase lives at:
13
14        `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`
15
16      (git remote `github.com/Khelsutra/badminton-highlight-indexer`, default branch
```

```
`master`).
17
18      **How to apply:** when a task references this repo's code (backend/worker,
pipeline/segmenters,
19      config.json, run_all_tests.py, etc.), `cd` to the khelsutra-guru path and work there ?
the stub dir
20      has none of it. Confirmed 2026-07-06 while fixing the wf294 silent-detector-failure
issue.
21
22      Repo conventions verified there: CI = ruff 0.15.16 (`ruff check .`), mypy 2.1.0 (`mypy
backend
23      training`; `mypy.ini` quarantines several modules incl. `backend.worker.__main__`,
`gemini`,
24      `yolo_hybrid` via `ignore_errors=True`), full suite via `python run_all_tests.py`,
coverage floor
25      `--cov-fail-under=70` (actual ~86%). Segmenters use a `Success`/`Failure` Result
contract
26      ([[efficient-timeouts-and-retries]] is unrelated).
27
```

6. assistant (2026-07-07T15:02:36.897Z)

Current branch `perf/validation-shared-decode-pass` is ahead of origin/master by 1 unpushed commit (the validation work from memory), plus two untracked dev-tool scripts. I must not disturb those. Let me fetch origin and inspect the relevant files before branching from `origin/master`.

7. user (2026-07-07T15:02:41.841Z)

```
---LOCAL COMMIT AHEAD OF MASTER---
d53851b perf(validators): decode scene_cut/blur/relevance frames in one shared pass
---FILES IN THAT COMMIT---
backend/pipeline/validators/base.py
backend/pipeline/validators/blur.py
backend/pipeline/validators/frame_sampler.py
backend/pipeline/validators/relevance.py
backend/pipeline/validators/scene_cut.py
docs/CODE_MAP.md
tests/test_frame_sampler.py
---MASTER HEAD---
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
```

8. user (2026-07-07T15:02:41.867Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
```

```
          worker that finished just past the deadline is rescued, not wasted ? then best-effort CANCELS
          the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
       3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
          ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` + native detector)
    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just computes.

    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole shim is unit-tested
    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-run`` and is
    owner-verified on the M7 real run.
    """

    from __future__ import annotations

    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional

    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
    from backend.infrastructure.execution_policy import (
        DEFAULT_POLICY,
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
    )
    from backend.storage import fetch, get_storage, parse_uri

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
```

```
67              project: Optional[str] = None,
68          ) -> str:
69              """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
70              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
71
72          @abstractmethod
73          def cancel_execution(self, execution: str) -> None:
74              """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
75              times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
76              May raise ? the caller treats every failure as best-effort (the original poll
timeout is
77              NEVER masked by a cancel error)."""
78
79
80      class GoogleCloudRunJobClient(CloudRunJobClient):
81          """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
82          a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84          def run_job(
85              self,
86              job_name: str,
87              argv: List[str],
88              *,
89              region: str,
90              project: Optional[str] = None,
91          ) -> str:
92              # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93              # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
94              # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95              if not project:
96                  raise RuntimeError(
97                      "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                      "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99                  )
100             try:
101                 from google.cloud import run_v2  # type: ignore[attr-defined]
102             except Exception as exc:  # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency)."
106                 ) from exc
107             client = (
108                 run_v2.JobsClient()
109             )  # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
112                 container_overrides=[
113                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114                 ]
115             )
116             op = client.run_job(
117                 request=run_v2.RunJobRequest(name=name, overrides=overrides)
118             )
```

```
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )

        def cancel_execution(self, execution: str) -> None:
            # run_job falls back to the literal string "execution" when the operation
metadata carries
            # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
            # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
            # google-cloud-run is absent).
            if "/executions/" not in (execution or ""):
                raise RuntimeError(
                    f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
                    "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
```

```
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._read_marker(done_uri),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                    # Live heartbeat (#482): each still-waiting tick reaches the caller so
                    # job.progress moves during the whole GPU run instead of freezing.
                    on_tick=on_wait,
                )
            except PolicyTimeout as timeout_exc:
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
```

```
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
            is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
            execution name so the operator can verify the state in the GCP console. A cancel
failure
            NEVER masks the original timeout."""
            try:
                late = self._read_marker(done_uri)
            except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as
absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
                    "? rescuing the completed result",
                    self._job_name,
                    execution,
                )
                return late
            try:
                self._client.cancel_execution(execution)
            except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
                cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
                LOG.warning(
                    "cloud-run: best-effort cancel of execution=%s failed: %s",
                    execution,
                    cancel_exc,
                    exc_info=True,
                )
            else:
                cancel_note = "cancel requested"
                LOG.info("cloud-run: requested cancel of execution=%s", execution)
            raise PolicyTimeout(
                f"{timeout_exc} [cloud-run job={self._job_name} "
                f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
```

```
292                 "state in the GCP console]"
293             ) from timeout_exc
294
295         # --- storage-backed completion signal (bucket-poll) ---
296         def _read_marker(self, done_uri: str) -> Optional[dict]:
297             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
298             ref = parse_uri(done_uri)
299             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
300                 return None
301             return (
302                 self._read_json(done_uri) or {}
303             )  # empty dict still terminates the poll (present == done)
304
305         def _read_json(self, uri: str) -> dict:
306             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
307             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
308             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
309             try:
310                 ref = parse_uri(uri)
311                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
312                     dst = os.path.join(td, ref.name or "obj.json")
313                     local = fetch(uri, dst, config=self._storage_config)
314                     with open(local, "r", encoding="utf-8") as f:
315                         data: Any = json.load(f)
316                 return data if isinstance(data, dict) else {}
317             except Exception:
318                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
319                 return {}
320
```

9. assistant (2026-07-07T15:02:49.147Z)

Now let me read the supporting pieces: the execution policy (polling primitives), the config model, and the existing tests for `cloud_run`.

10. user (2026-07-07T15:02:50.145Z)

```
1       """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3       Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4       - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7       - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10      **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11      waiting (a fake clock advances instantly).
12
13      **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14      transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15      ``execution.policy`` logger to watch the poll trail.
16      """
17
```

```
    from __future__ import annotations

    import logging
    import threading
    import time as _time
    from dataclasses import asdict, dataclass
    from enum import Enum
    from typing import Callable, Optional, TypeVar

    T = TypeVar("T")

    #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
    LOG = logging.getLogger("execution.policy")


    class WaitMode(str, Enum):
        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""

        WAIT_AND_RESUME = "wait_and_resume"  # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
        ABORT = "abort"  # one attempt; on hang/fail, stop immediately
        BLOCK = "block"  # bounded in-process blocking wait (no reschedule)


    class PolicyTimeout(TimeoutError):
        """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""


    @dataclass(frozen=True)
    class ExecutionPolicy:
        """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""

        mode: WaitMode = WaitMode.WAIT_AND_RESUME
        poll_every_n_sec: float = 10.0
        op_timeout_sec: float = (
            120.0  # HARD per-attempt deadline ? the hang-killer that was missing
        )
        max_wait_sec: float = 1800.0  # total wall budget for a poll loop
        max_retries: int = 5  # retries (=> max_retries+1 attempts) for failures/hangs
        backoff_base_sec: float = 3.0  # exponential: backoff_base * 2**attempt
        on_timeout: str = "reschedule"  # reschedule | partial | fail  (consumed by P2/P3)
        resumable: bool = True

        def to_dict(self) -> dict:
            d = asdict(self)
            d["mode"] = self.mode.value
            return d

        @classmethod
        def from_dict(cls, d: dict) -> "ExecutionPolicy":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            kw = {k: v for k, v in d.items() if k in known}
            if "mode" in kw and not isinstance(kw["mode"], WaitMode):
                kw["mode"] = WaitMode(kw["mode"])
            return cls(**kw)


    #: The sane default (also what a job gets if it declares no policy).
    DEFAULT_POLICY = ExecutionPolicy()


```

```
    def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
        """Deterministic exponential backoff (no jitter ? reproducible tests)."""
        return policy.backoff_base_sec * (2**attempt)


    def run_bounded(
        fn: Callable[[], T],
        policy: ExecutionPolicy = DEFAULT_POLICY,
        *,
        label: str = "op",
        logger: Optional[logging.Logger] = None,
        sleep: Callable[[float], None] = _time.sleep,
    ) -> T:
        """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.

        Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
        last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
        """
        log = logger or LOG
        attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
        last_exc: Optional[BaseException] = None
        for attempt in range(attempts):
            box: dict = {}
            done = threading.Event()

            # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
            # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
            def _runner(_box: dict = box, _done: threading.Event = done) -> None:
                try:
                    _box["val"] = fn()
                except (
                    BaseException
                ) as e:  # relay ANY error (incl. timeouts) to the caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info(
                        "%s: attempt %d/%d failed: %s",
                        label,
                        attempt + 1,
                        attempts,
                        last_exc,
                    )
                else:
                    if attempt:
                        log.info(
                            "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
                        )
                    return box["val"]  # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(
                    f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
                )
                log.warning(
                    "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
```

```
                    label,
                    attempt + 1,
                    attempts,
                    policy.op_timeout_sec,
                )
            if attempt < attempts - 1:
                wait = _backoff_sec(policy, attempt)
                log.debug("%s: backing off %.1fs before retry", label, wait)
                sleep(wait)
        assert last_exc is not None
        log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
        raise last_exc


    def poll_until(
        check: Callable[[], Optional[T]],
        policy: ExecutionPolicy = DEFAULT_POLICY,
        *,
        label: str = "op",
        logger: Optional[logging.Logger] = None,
        sleep: Callable[[float], None] = _time.sleep,
        monotonic: Callable[[], float] = _time.monotonic,
        on_tick: Optional[Callable[[float], None]] = None,
    ) -> T:
        """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
        ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.

        Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
        ``on_tick(elapsed_sec)`` (optional) fires once per still-waiting iteration ? the
caller's live
        progress heartbeat (#482). Best-effort by contract: a raising callback is swallowed
        (DEBUG-logged), never allowed to kill the wait it reports on.
        """
        log = logger or LOG
        start = monotonic()
        polls = 0
        log.info(
            "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
            label,
            policy.poll_every_n_sec,
            policy.max_wait_sec,
        )
        while True:
            result = check()
            if result is not None:
                log.info(
                    "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
                )
                return result
            elapsed = monotonic() - start
            if elapsed >= policy.max_wait_sec:
                log.warning(
                    "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
                )
                raise PolicyTimeout(
                    f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
                )
            polls += 1
            log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
            if on_tick is not None:
                try:
                    on_tick(elapsed)
```

```
199                 except Exception:  # noqa: BLE001 ? progress reporting must never kill the
wait
200                     log.debug("%s: on_tick callback failed", label, exc_info=True)
201             sleep(policy.poll_every_n_sec)
202
```

11. user (2026-07-07T15:02:54.652Z)

```
=== test files mentioning cloud_run ===
tests/test_api_cloud_dispatch.py
tests/test_billing.py
tests/test_compute_backend.py
tests/test_config_deployment.py
tests/test_deploy_manifests.py
tests/test_drain_concurrency.py
tests/test_restart_recovery.py
tests/test_startup_checks.py
tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_billing.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_compute_backend.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_config_deployment.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_deploy_manifests.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_drain_concurrency.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_restart_recovery.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_startup_checks.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_startup_checks.cpython-313.pyc
=== config models remote_poll ===
483:    remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
```

12. assistant (2026-07-07T15:02:58.224Z)

Let me look at the primary test file for the compute backend and the config model context around the poll setting.

13. user (2026-07-07T15:03:00.114Z)

```
1       """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3       All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4       worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5       ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6       selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7       """
8
9       import json
10      import os
11      import tempfile
12
13      import pytest
14
15      from backend.compute import ComputeJobSpec, get_compute_backend
16      from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
17      from backend.config import AppConfig
18      from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
19
20      # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
21      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
22
23
```

```
    class _FakeRunClient(CloudRunJobClient):
        """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
        exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""

        def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
            self.calls: list[dict] = []
            self.video_id, self.segment_count, self.returncode = (
                video_id,
                segment_count,
                returncode,
            )

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(
                {"job": job_name, "argv": list(argv), "region": region, "project": project}
            )
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            outputs = os.path.join(out_uri, "outputs", self.video_id)
            os.makedirs(outputs, exist_ok=True)
            with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                json.dump(
                    {
                        "video_id": self.video_id,
                        "segment_count": self.segment_count,
                        "status": "success",
                    },
                    f,
                )
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump(
                    {
                        "video_id": self.video_id,
                        "returncode": self.returncode,
                        "segment_count": self.segment_count,
                        "status": "success",
                    },
                    f,
                )
            return "exec-123"

        def cancel_execution(self, execution):
            raise AssertionError(
                "cancel must never fire on the happy path"
            )  # pragma: no cover


    class _NoopRunClient(CloudRunJobClient):
        """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
        directly (a gs:// out_uri has no local dir to makedirs)."""

        def __init__(self):
            self.calls: list[dict] = []
            self.cancelled: list[str] = []

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(
                {"job": job_name, "argv": list(argv), "region": region, "project": project}
            )
            return "exec"

```

```
        def cancel_execution(self, execution):
            self.cancelled.append(execution)


    class _NeverDoneClient(CloudRunJobClient):
        """Simulates a still-running execution: dispatch succeeds (returning a fully-qualified
        execution name) but NO done-marker is ever written, so the bucket-poll must time out."""

        EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-slow"

        def __init__(self, *, cancel_error: Exception | None = None):
            self.calls: list[dict] = []
            self.cancelled: list[str] = []
            self._cancel_error = cancel_error

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(
                {"job": job_name, "argv": list(argv), "region": region, "project": project}
            )
            return self.EXEC_NAME

        def cancel_execution(self, execution):
            self.cancelled.append(execution)
            if self._cancel_error is not None:
                raise self._cancel_error


    # ---------------------------------------------------------------- factory (default-OFF)


    def _cfg(compute_target="", storage_backend="local"):
        return AppConfig.model_validate(
            {
                "deployment": {"compute_target": compute_target},
                "storage": {"backend": storage_backend},
            }
        )


    def test_factory_is_none_for_local_targets():
        """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process path)."""
        assert get_compute_backend(_cfg("")) is None
        assert get_compute_backend(_cfg("local_gpu")) is None
        assert get_compute_backend(_cfg("cpu_mock")) is None


    def test_factory_returns_cloud_run_for_cloud_target():
        backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
        assert isinstance(backend, CloudRunCompute)


    # ------------------------------------------------------ real client: missing-project guard


    def test_real_client_rejects_missing_project():
        """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is unset ?
        a None/empty project would build a bogus "projects/None/..." job path (job_path requires a
```

```
        real str). The guard runs BEFORE the google-cloud-run import, so it is exercisable
in CI
        (the SDK is intentionally not a test dependency)."""
        from backend.compute.cloud_run import GoogleCloudRunJobClient

        client = GoogleCloudRunJobClient()
        for missing in (None, ""):
            with pytest.raises(RuntimeError, match="GOOGLE_CLOUD_PROJECT"):
                client.run_job(
                    "rally-worker", ["infer"], region="asia-south1", project=missing
                )


    def test_real_client_cancel_rejects_unresolvable_execution_name():
        """cancel_execution must fail loudly on a non-resolvable execution name ? run_job
falls back
        to the literal "execution" when the operation metadata carries no name, and
cancelling that
        would be a bogus API call. The guard runs BEFORE the google-cloud-run import
(mirrors the
        project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
        from backend.compute.cloud_run import GoogleCloudRunJobClient

        client = GoogleCloudRunJobClient()
        for bogus in ("", "execution", "exec-123"):
            with pytest.raises(RuntimeError, match="fully-qualified"):
                client.cancel_execution(bogus)


    # ------------------------------------------------------------ CloudRunCompute dispatch
+ poll


    def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
        client = _FakeRunClient(video_id="abc123", segment_count=3)
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            project="proj",
            storage_config={},
            policy=_FAST,
            token="tok",
        )
        out = str(tmp_path / "bucket")
        result = backend.run_infer(
            ComputeJobSpec(
                video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
            )
        )

        assert result.returncode == 0
        assert result.video_id == "abc123"
        assert result.outputs_uri == f"{out}/outputs/abc123/"
        assert result.report.get("segment_count") == 3
        # exactly one Cloud Run Job execution was triggered, in the right region/project
        assert len(client.calls) == 1
        assert (
            client.calls[0]["job"] == "rally-worker"
            and client.calls[0]["region"] == "asia-south1"
        )


    def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
```

```
        """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
        native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
        client = _FakeRunClient()
        backend = CloudRunCompute(
            run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
        )
        backend.run_infer(
            ComputeJobSpec(
                video_uri="gs://b/v.mp4",
                out_uri=str(tmp_path / "bk"),
                segmenter="wasb_hybrid",
                config_overrides=("sport=tennis",),
            )
        )

        argv = client.calls[0]["argv"]
        assert argv[0] == "infer"
        assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        # the container-inline overrides come last (win) and the user override is carried
through
        joined = " ".join(argv)
        assert "deployment.compute_target=local_gpu" in joined
        assert "indexing.detector_impl=native" in joined
        assert "sport=tennis" in joined
        # user override precedes the inline-forcing overrides in --set order
        assert joined.index("sport=tennis") < joined.index(
            "deployment.compute_target=local_gpu"
        )


    def test_unique_token_per_call_when_not_injected(tmp_path):
        """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
        backend.run_infer(
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
        )
        backend.run_infer(
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
        )
        done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
        done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
        assert done1 != done2


    def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
        """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
        pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
        identity passthrough). Mimic a cloud backend that rejects dst=None."""
        from backend.compute import cloud_run as cr

        captured = {}

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            captured["dst"] = dst
            if dst is None:
                raise ValueError(
                    "cloud backend requires a dst path"
                )  # mirror GCSStorage/S3Storage
```

```
            with open(dst, "w", encoding="utf-8") as f:
                f.write('{"video_id": "v", "returncode": 0}')
            return dst

        monkeypatch.setattr(cr, "fetch", fake_fetch)
        backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
        data = backend._read_json("gs://b/x.json")
        assert (
            captured["dst"] is not None
        )  # the fix: a real dst is always passed (no silent {})
        assert data == {"video_id": "v", "returncode": 0}
        assert not os.path.exists(os.path.dirname(captured["dst"]))


    def test_cloud_run_polls_until_marker_appears(monkeypatch):
        """The bucket-poll WAIT path (D7 scale-to-zero cold start): the marker is ABSENT for
the first
        polls then appears ? poll_until must LOOP, not resolve on poll 0. The synchronous
fake elsewhere
        never exercises this; here exists() is False twice then True."""
        from backend.compute import cloud_run as cr

        exists_calls = {"n": 0}

        class _FakeStore:
            def exists(self, ref):
                exists_calls["n"] += 1
                return exists_calls["n"] >= 3  # absent for polls 1-2, present on poll 3

        monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            if dst is None:
                dst = os.path.join(tempfile.mkdtemp(prefix="cr-poll-"), "m.json")
            with open(dst, "w", encoding="utf-8") as f:
                f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
            return dst

        monkeypatch.setattr(cr, "fetch", fake_fetch)
        backend = cr.CloudRunCompute(
            run_client=_NoopRunClient(),
            job_name="j",
            region="r",
            token="t",
            policy=ExecutionPolicy(
                poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0
            ),
        )
        result = backend.run_infer(
            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
        )
        assert result.video_id == "vlate"
        assert (
            exists_calls["n"] >= 3
        )  # >=1 absent poll occurred before the marker appeared (loop ran)


    # ---------------------------------------- poll timeout: rescue-or-cancel (R1-21 GPU
cost leak)

    # max_wait_sec=0.0 ? poll_until gives up after its FIRST absent check (no real waiting).
    _TIMEOUT_NOW = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0,
max_wait_sec=0.0)


```

```
    def test_poll_timeout_cancels_execution_and_raises(tmp_path):
        """Poll deadline + no marker ? the still-running execution is cancelled (with the
EXACT name
        run_job returned) and PolicyTimeout surfaces carrying that name for the operator."""
        client = _NeverDoneClient()
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            policy=_TIMEOUT_NOW,
            token="tok",
        )
        with pytest.raises(PolicyTimeout) as exc:
            backend.run_infer(
                ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
            )
        assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # cancel got the right name
        msg = str(exc.value)
        assert _NeverDoneClient.EXEC_NAME in msg  # operator can find the execution in GCP
        assert "cancel requested" in msg
        assert "max_wait_sec" in msg  # the original timeout text is preserved, not masked


    def test_poll_timeout_cancel_failure_still_raises_policy_timeout(tmp_path):
        """Best-effort means BEST-EFFORT: a cancel API failure must never mask the original
timeout ?
        the caller (main.py's dispatch handler) still sees PolicyTimeout, with the failure
noted."""
        client = _NeverDoneClient(cancel_error=RuntimeError("cancel API unreachable"))
        backend = CloudRunCompute(
            run_client=client,
            job_name="rally-worker",
            region="asia-south1",
            policy=_TIMEOUT_NOW,
            token="tok",
        )
        with pytest.raises(PolicyTimeout) as exc:
            backend.run_infer(
                ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
            )
        assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # the attempt WAS made
        msg = str(exc.value)
        assert _NeverDoneClient.EXEC_NAME in msg
        assert "cancel API unreachable" in msg  # the cancel failure is surfaced, not
swallowed
        assert "max_wait_sec" in msg  # ...alongside the original timeout text


    def test_poll_timeout_late_marker_rescues_result(monkeypatch):
        """A slow-but-healthy worker that finishes JUST past the poll deadline
(max_wait_sec=1800 can
        undercut the job's --task-timeout=3600) is rescued by the ONE final post-timeout
marker check:
        the completed GPU result is ingested, nothing is cancelled, no timeout surfaces."""
        from backend.compute import cloud_run as cr

        exists_calls = {"n": 0}

        class _FakeStore:
            def exists(self, ref):
                exists_calls["n"] += 1
                return exists_calls["n"] >= 2  # absent for the poll, present on the final
check

        monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
```

```
380
381         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
382             if dst is None:
383                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-late-"), "m.json")
384             with open(dst, "w", encoding="utf-8") as f:
385                 f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
386             return dst
387
388         monkeypatch.setattr(cr, "fetch", fake_fetch)
389         client = _NoopRunClient()
390         backend = cr.CloudRunCompute(
391             run_client=client, job_name="j", region="r", token="t", policy=_TIMEOUT_NOW
392         )
393         result = backend.run_infer(
394             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
395         )
396         assert result.returncode == 0
397         assert result.video_id == "vlate"  # the late result is ingested, not wasted
398         assert client.cancelled == []  # a finished execution is never cancelled
399         assert exists_calls["n"] == 2  # exactly ONE extra post-timeout check
400
401
402     # ---------------------------------------------------- worker wiring: dispatch branch +
done-marker
403
404
405     def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
406         """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
407         from backend.compute.base import ComputeResult
408         from backend.worker import __main__ as wmain
409
410         seen = {}
411
412         class _FakeBackend:
413             def run_infer(self, spec):
414                 seen["spec"] = spec
415                 return ComputeResult(
416                     returncode=0, video_id="vidX", outputs_uri="gs://b/outputs/vidX/"
417                 )
418
419         monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
420         # cloud-serving startup checks require the AI-handoff key (AI_HANDOFF_KEY_MISSING is
fatal).
421         monkeypatch.setenv("GEMINI_API_KEY", "test-key")
422         cfg = tmp_path / "c.json"
423         cfg.write_text(
424             '{"deployment": {"profile": "cloud-serving"}, "storage": {"backend": "gcs"}}'
425         )
426         rc = wmain.cmd_infer(
427             wmain.build_parser().parse_args(
428                 [
429                     "infer",
430                     "--video-uri",
431                     "gs://b/videos/clip.mp4",
432                     "--out-uri",
433                     "gs://b",
434                     "--config",
435                     str(cfg),
436                     "--segmenter",
437                     "wasb_hybrid",
438                 ]
439             )
440         )
441         assert rc == 0
```

```
        assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
        assert seen["spec"].segmenter == "wasb_hybrid"


    def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
        """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
        remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
        from backend.worker import __main__ as wmain

        class _OK:
            passed = True
            validator_name = "fake"
            message = "ok"

            def model_dump(self):
                return {"validator_name": "fake", "passed": True, "message": "ok"}

        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OK()], {}),
        )
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setattr(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
            lambda p: 30.0,
        )

        cfg = tmp_path / "c.json"
        cfg.write_text(
            json.dumps(
                {
                    "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
                    "output": {"output_dir": str(tmp_path / "o")},
                }
            )
        )
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        out = tmp_path / "bucket"
        done = str(out / "_dispatch" / "tok.done")

        rc = wmain.cmd_infer(
            wmain.build_parser().parse_args(
                [
                    "infer",
                    "--video-uri",
                    str(video),
                    "--out-uri",
                    str(out),
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--segmenter",
                    "wasb_hybrid",
                    "--done-uri",
                    done,
                ]
            )
        )
        assert rc == 0
        marker = json.loads(open(done, encoding="utf-8").read())
```

```
        assert marker["returncode"] == 0 and marker["status"] == "success"
        assert marker["video_id"] and marker["segment_count"] >= 1
        # CP-rebuild P3 (#485): the report records the SOURCE the detection ran on, so a
        # restart-wiped control plane re-hydrating from outputs/<md5>/ can restore the video
        # row's truthful path (not just the segments).
        report_path = out / "outputs" / marker["video_id"] / "report.json"
        report = json.loads(report_path.read_text(encoding="utf-8"))
        assert report["video_uri"] == str(video)


    def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
        """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
        from backend.worker import __main__ as wmain

        class _OK:
            passed = True
            validator_name = "fake"
            message = "ok"

            def model_dump(self):
                return {"validator_name": "fake", "passed": True, "message": "ok"}

        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OK()], {}),
        )
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setattr(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
            lambda p: 30.0,
        )
        cfg = tmp_path / "c.json"
        cfg.write_text(
            json.dumps(
                {
                    "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
                    "output": {"output_dir": str(tmp_path / "o")},
                }
            )
        )
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        out = tmp_path / "bucket"
        rc = wmain.cmd_infer(
            wmain.build_parser().parse_args(
                [
                    "infer",
                    "--video-uri",
                    str(video),
                    "--out-uri",
                    str(out),
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--segmenter",
                    "wasb_hybrid",
                ]
            )
        )
        assert rc == 0
        assert not (out / "_dispatch").exists()  # no marker dir when --done-uri is omitted

```

```
569
570     def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
571         """The container side: a worker-side FAILURE (validation rc=2) with --done-uri
writes a marker
572         carrying the REAL rc, so a remote dispatcher's bucket-poll terminates FAST (not a
30-min hang)."""
573         from backend.worker import __main__ as wmain
574
575         class _BAD:
576             passed = False
577             validator_name = "blur"
578             message = "blurry"
579
580             def model_dump(self):
581                 return {"validator_name": "blur", "passed": False, "message": "blurry"}
582
583         monkeypatch.setattr(
584             wmain.validator_registry,
585             "run_all_with_context",
586             lambda path, cfg: ([_BAD()], {}),
587         )
588         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
589         cfg = tmp_path / "c.json"
590         cfg.write_text(
591             json.dumps(
592                 {
593                     "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
594                     "output": {"output_dir": str(tmp_path / "o")},
595                 }
596             )
597         )
598         video = tmp_path / "in.mp4"
599         video.write_bytes(b"FAKEVIDEO")
600         out = tmp_path / "bucket"
601         done = str(out / "_dispatch" / "tok.done")
602
603         rc = wmain.cmd_infer(
604             wmain.build_parser().parse_args(
605                 [
606                     "infer",
607                     "--video-uri",
608                     str(video),
609                     "--out-uri",
610                     str(out),
611                     "--config",
612                     str(cfg),
613                     "--scratch",
614                     str(tmp_path / "s"),
615                     "--segmenter",
616                     "wasb_hybrid",
617                     "--done-uri",
618                     done,
619                 ]
620             )
621         )
622         assert rc == 2  # validation failure
623         marker = json.loads(open(done, encoding="utf-8").read())
624         assert marker["returncode"] == 2 and marker["status"] == "failed"
625         assert marker[
626             "video_id"
627         ]  # carries the id so the dispatcher's poll terminates (rc != 0 ? failed)
628
629
630     def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
631         """DEFAULT-OFF: a worker failure with NO --done-uri writes no marker (the unchanged
```

```
local run)."""
632         from backend.worker import __main__ as wmain
633
634         class _BAD:
635             passed = False
636             validator_name = "blur"
637             message = "blurry"
638
639             def model_dump(self):
640                 return {"validator_name": "blur", "passed": False, "message": "blurry"}
641
642         monkeypatch.setattr(
643             wmain.validator_registry,
644             "run_all_with_context",
645             lambda path, cfg: ([_BAD()], {}),
646         )
647         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
648         cfg = tmp_path / "c.json"
649         cfg.write_text(
650             json.dumps(
651                 {
652                     "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
653                     "output": {"output_dir": str(tmp_path / "o")},
654                 }
655             )
656         )
657         video = tmp_path / "in.mp4"
658         video.write_bytes(b"FAKEVIDEO")
659         out = tmp_path / "bucket"
660         rc = wmain.cmd_infer(
661             wmain.build_parser().parse_args(
662                 [
663                     "infer",
664                     "--video-uri",
665                     str(video),
666                     "--out-uri",
667                     str(out),
668                     "--config",
669                     str(cfg),
670                     "--scratch",
671                     str(tmp_path / "s"),
672                     "--segmenter",
673                     "wasb_hybrid",
674                 ]
675             )
676         )
677         assert rc == 2
678         assert not (out / "_dispatch").exists()
679
680
681     # ---------------------------------------------------------- Dockerfile structural guard
(no build)
682
683
684     def test_dockerfile_has_required_directives():
685         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
structurally:
686         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
worker entrypoint."""
687         from pathlib import Path
688
689         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
690         assert "FROM nvidia/cuda" in text  # GPU base (CUDA)
691         assert "ffmpeg" in text  # stitch/decode
692         assert "fonts-dejavu" in text  # the watermark filter font
```

```
693         # The #506 modernized WASB stack (L4-validated): torch 2.6 cu124 + PyNvVideoCodec,
and
694         # the NVDEC/NVENC driver userspace libs PyNvVideoCodec dlopens (compute-only GPU
hosts
695         # inject neither) ? a silent drop of any of these bricks
decode_backend=nvdec_resident.
696         assert "torch==2.6.0" in text
697         assert "download.pytorch.org/whl/cu124" in text
698         assert "PyNvVideoCodec" in text
699         assert "libnvcuvid.so.1" in text and "libnvidia-encode.so.1" in text
700         assert "torch==1.11.0+cu113" not in text  # the old stack is gone, not co-installed
701         assert "WASB-SBDT" in text  # the shuttle detector repo
702         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
703         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
704         # The cloud worker reads the gs:// input + writes the gs:// outputs/done-marker
through
705         # backend.storage (storage-gcs); and when this image is reused as the deployed
control plane it
706         # dispatches the L4 Job via google.cloud.run_v2 (cloud-run) and verifies Cf-Access
JWTs when
707         # edge-auth is on (auth ? PyJWT). A bare `pip install .` boots but dies on the first
gs:// fetch
708         # (ModuleNotFoundError: google.cloud), on dispatch (google-cloud-run missing), or on
the first
709         # edge-authed request (jwt missing) ? real Cloud Run failures the mock-client tests
can't see.
710         # Lock the extras in structurally.
711         assert ".[storage-gcs,cloud-run,auth]" in text
712
713
714     def test_dockerfile_uses_micromamba_not_anaconda_tos():
715         """P3a: the WASB env is built with micromamba + conda-forge (BSD, no ToS), NOT
Miniconda +
716         `conda tos accept` (the ?200-employee Anaconda Terms-of-Service trap,
PACKAGING_PLAN.md §3).
717         Locked structurally so a revert to the ToS path fails CI; the env path (WASB_PYTHON)
is unchanged."""
718         from pathlib import Path
719
720         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
721         assert "micromamba" in text  # the conda-forge env builder
722         assert "conda-forge" in text  # the BSD, ToS-free channel
723         assert (
724             "MAMBA_ROOT_PREFIX=/opt/conda" in text
725         )  # keeps the env at /opt/conda/envs/wasb
726         assert "Miniconda3" not in text  # the Anaconda installer is gone
727         assert "conda tos accept" not in text  # no ToS acceptance needed (conda-forge)
728
729
730     def test_remote_poll_budget_comes_from_config_unless_policy_injected():
731         """CP-rebuild §4.3: deployment.remote_poll_max_wait_sec drives the dispatch poll
budget
732         (default 3900s > the Job's --task-timeout 3600s ? the old 1800s default could
abandon a
733         slow-but-healthy paid GPU run), and an explicitly injected policy always wins."""
734         from backend.infrastructure.execution_policy import DEFAULT_POLICY
735
736         cfg = AppConfig()
737         cfg.deployment.compute_target = "cloud_run"
738
739         be = get_compute_backend(cfg, run_client=_FakeRunClient())
740         assert be is not None
741         assert be._policy.max_wait_sec == 3900.0
742         # Only the budget is derived ? the other resilience knobs stay at their defaults.
743         assert be._policy.poll_every_n_sec == DEFAULT_POLICY.poll_every_n_sec
```

```
744         assert be._policy.op_timeout_sec == DEFAULT_POLICY.op_timeout_sec
745
746         cfg.deployment.remote_poll_max_wait_sec = 42.0
747         assert get_compute_backend(cfg, run_client=_FakeRunClient())._policy.max_wait_sec ==
42.0
748
749         # An injected policy is never overridden by config.
750         be = get_compute_backend(cfg, run_client=_FakeRunClient(), policy=_FAST)
751         assert be._policy.max_wait_sec == _FAST.max_wait_sec
752
753
754     def test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
755         """The #482 heartbeat plumbing: the on_wait callback handed to run_infer must be the
756         exact on_tick the bucket-poll drives (cloud_run threads it through poll_until)."""
757         from backend.compute import cloud_run as cr
758
759         seen = {}
760
761         def fake_poll(check, policy, *, label="op", logger=None, on_tick=None, **kw):
762             seen["on_tick"] = on_tick
763             return {"video_id": "vidCloud", "returncode": 0}
764
765         monkeypatch.setattr(cr, "poll_until", fake_poll)
766         client = _FakeRunClient()  # still writes report.json so the post-marker read
succeeds
767         backend = CloudRunCompute(
768             run_client=client,
769             job_name="rally-worker",
770             region="asia-southeast1",
771             storage_config={},
772             policy=_FAST,
773             token="tok",
774         )
775
776         def heartbeat(_elapsed):
777             pass
778
779         result = backend.run_infer(
780             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk")),
781             on_wait=heartbeat,
782         )
783         assert seen["on_tick"] is heartbeat
784         assert result.video_id == "vidCloud"
785
```

14. user (2026-07-07T15:03:00.449Z)

```
450         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451         gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
460         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
461         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
462         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
```

```
knobs
463         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD §4.3).
Must sit
479         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493         inserts a real, calibrated version and points ``pricebook_version`` at it.
494
495         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
496         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
497         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
498
499         enabled: bool = False
500         pricebook_version: str = (
501             "placeholder-v0"  # the seeded $0 version; owner flips to a real one
502         )
503         fx_inr_per_usd: float = 83.0  # USD?INR display rate (Bangalore market)
504         margin: float = 0.0  # resale markup (?0); 0 = bill at cost
505
506
507     class FlywheelConfig(BaseModel):
```

```
508         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
509         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
510         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
511         the owned model climbs toward the D11 ship floor.
512
513         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
514         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
515         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
516         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
517         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
518         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
519         owner/counsel/GPU scope. Pairs with M9-consent."""
```

15. assistant (2026-07-07T15:03:41.512Z)

Now let me understand the caller path ? how `run_infer`'s result/exceptions become the "cloud dispatch error (rc 4)" the task references, and the returncode conventions.

16. user (2026-07-07T15:03:43.217Z)

```
Found 7 files
backend\orchestration.py
backend\worker\__main__.py
backend\api\job_worker.py
backend\infrastructure\execution_policy.py
backend\compute\cloud_run.py
backend\compute\base.py
backend\providers\gemini.py
```

17. user (2026-07-07T15:03:44.261Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py-187-        segmenter=args.segmenter,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py-188-        config_overrides=tuple(args.set or ()),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py-189-    )
backend\worker\__main__.py-190-    try:
backend\worker\__main__.py:191:        result = compute.run_infer(spec)
backend\worker\__main__.py-192-    except PolicyTimeout as e:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py-193-        logger.warning(
backend\worker\__main__.py-194-            "[worker] remote compute did not complete (no marker
within budget): %s", e
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py-195-        )
```

18. assistant (2026-07-07T15:03:47.493Z)

Let me read the full dispatch handler in the worker `__main__.py` around this call, and check the rc-4 path.

19. user (2026-07-07T15:03:48.252Z)

```
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
```

```
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
180         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
181         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
182         from backend.infrastructure.execution_policy import PolicyTimeout
183
184         spec = ComputeJobSpec(
185             video_uri=args.video_uri,
186             out_uri=args.out_uri,
187             segmenter=args.segmenter,
188             config_overrides=tuple(args.set or ()),
189         )
190         try:
191             result = compute.run_infer(spec)
192         except PolicyTimeout as e:
193             logger.warning(
194                 "[worker] remote compute did not complete (no marker within budget): %s", e
195             )
196             return 4
197         logger.info(
198             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
199             result.video_id,
200             result.returncode,
201             result.outputs_uri,
202         )
203         return result.returncode
204
205
206     def _write_done_marker(
207         done_uri: str,
208         video_id: str,
209         returncode: int,
210         segment_count: int,
211         status: str,
212         storage_cfg: dict,
213         scratch: str,
214     ) -> None:
215         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
216         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
```

```
return."""
217         try:
218             marker = {
219                 "video_id": video_id,
220                 "returncode": returncode,
221                 "segment_count": segment_count,
222                 "status": status,
223             }
224             local = os.path.join(scratch, "_done.json")
225             with open(local, "w", encoding="utf-8") as f:
226                 json.dump(marker, f)
227             storage.put(local, done_uri, config=storage_cfg)
228             logger.info("[worker] wrote completion marker -> %s", done_uri)
229         except Exception as e:  # never fail a good run because the marker push hiccuped
230             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
231
232
233     def cmd_infer(args: argparse.Namespace) -> int:
234         config = load_config(args.config) if args.config else load_config()
235         _apply_overrides(config, args.set)
236         if not _run_startup_checks(config):
237             return 2
238
239         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
240         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
241         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
242         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
243         compute = get_compute_backend(config)
244         if compute is not None:
245             return _dispatch_remote(compute, args)
246
247         storage_cfg = config.storage.model_dump()
248
249         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
```

20. assistant (2026-07-07T15:05:00.406Z)

`_dispatch_remote` catches `PolicyTimeout` ? rc 4; a non-`PolicyTimeout` exception from `run_infer` would escape as a raw traceback. Let me check the returncode conventions to pick a distinct, clearly-labelled code for the new fast-fail.

21. user (2026-07-07T15:05:02.055Z)

```
backend\worker\__main__.py:156:    cleanly (rc 2) rather than run an expensive job under an
incoherent profile. A true no-op
backend\worker\__main__.py:180:    poll exhaust its budget ? ``PolicyTimeout``. Catch it and
return a CLEAN rc 4 (remote dispatch
backend\worker\__main__.py:196:        return 4
backend\worker\__main__.py:237:        return 2
backend\worker\__main__.py:386:    # terminates (carries video_id ? the dispatcher's output path
+ rc=0). Worker FAILURES (rc 2/3)
backend\worker\__main__.py:526:        return 2
backend\worker\__main__.py:547:        return 2
backend\worker\__main__.py:556:            return 2
backend\worker\__main__.py:631:        return 2
```

22. user (2026-07-07T15:05:04.097Z)

```
backend\api\job_worker.py:74:    may run ``deployment.max_concurrent_remote_jobs`` drain workers
so I/O-bound remote dispatch/poll
```

```
backend\compute\base.py:35:    """The terminal state of a dispatched job. ``returncode`` mirrors
the worker's exit code
backend\compute\base.py:36:    (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 =
remote dispatch timeout or an
backend\compute\base.py:40:    returncode: int
backend\eval\ablation.py:572:        raise SystemExit(
backend\compute\cloud_run.py:14:  3. **Read** the marker (it carries ``video_id`` + the worker's
returncode) ? fetch
backend\compute\cloud_run.py:225:        # A MISSING returncode defaults to FAILURE (1), never
success ? a present-but-unreadable /
backend\compute\cloud_run.py:227:        rc = int(marker.get("returncode", 1))
backend\compute\cloud_run.py:241:            returncode=rc,
backend\eval\calibrate_wasb.py:92:        raise SystemExit(f"no usable golden rallies in
{labels_csv}")
backend\eval\eval_partial_labels.py:102:        raise SystemExit(f"no usable human rallies in
{labels_csv}")
backend\eval\eval_partial_labels.py:209:        raise SystemExit(f"no usable human rallies in
{labels_csv}")
backend\eval\calibrate_local.py:94:            raise SystemExit(f"no usable labels in
{s['labels']} for {s.get('name')}")
backend\worker\__main__.py:156:    cleanly (rc 2) rather than run an expensive job under an
incoherent profile. A true no-op
backend\worker\__main__.py:180:    poll exhaust its budget ? ``PolicyTimeout``. Catch it and
return a CLEAN rc 4 (remote dispatch
backend\worker\__main__.py:200:        result.returncode,
backend\worker\__main__.py:203:    return result.returncode
backend\worker\__main__.py:209:    returncode: int,
backend\worker\__main__.py:220:            "returncode": returncode,
backend\worker\__main__.py:385:    # M6: a remote dispatcher passes --done-uri; write the
SUCCESS completion marker so its bucket-poll
backend\worker\__main__.py:386:    # terminates (carries video_id ? the dispatcher's output path
+ rc=0). Worker FAILURES (rc 2/3)
backend\worker\__main__.py:662:    rc = subprocess.run(cmd).returncode
backend\eval\experiment.py:234:          * ``"require"``          ? a fusion file lacking audio
raises ``SystemExit`` (via
backend\eval\experiment.py:252:            raise SystemExit(no_audio_error.format(path=path))
backend\eval\fusion_audio.py:195:            raise SystemExit(f"need >=2 augmented feature files
in {args.features_dir}")
backend\eval\fusion_compare.py:105:        raise SystemExit(
backend\orchestration.py:50:# rate-fragile; a remote dispatch is a bucket-poll and never holds
this lane, so N remote jobs overlap
backend\orchestration.py:589:    if result.returncode != 0:
backend\orchestration.py:590:        return _failed(f"cloud job failed (returncode
{result.returncode})."", True)
backend\eval\export_wasb_segments.py:235:            raise SystemExit(f"no usable golden rallies
in {labels_csv}")
backend\eval\export_wasb_segments.py:318:            raise SystemExit("--slice requires
--video")
backend\eval\golden_manifest.py:434:    raise SystemExit(main())
backend\eval\gemini_refine.py:140:        raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:232:        raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:354:        raise SystemExit("--trajectory is required unless
--full-video")
backend\eval\golden_fixtures.py:1035:    raise SystemExit(main())
backend\eval\rally_seg_eval.py:570:    raise SystemExit(f"unknown preset '{name}'; choices:
{sorted(PRESETS)}")
backend\pipeline\audio\hit_detector.py:136:    if win % 2 == 0:
backend\pipeline\detectors\native_wasb_runner.py:166:        return res.returncode == 0 and
"cuda True" in (res.stdout or "")
backend\pipeline\detectors\native_wasb_runner.py:236:        if res.returncode != 0:

[Showing results with pagination = limit: 40]
```

23. assistant (2026-07-07T15:05:09.459Z)

Let me read the returncode legend in `base.py` and the orchestration consumer, then I'll have full context to design.

24. user (2026-07-07T15:05:10.983Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Callable, Optional
18
19
20      @dataclass(frozen=True)
21      class ComputeJobSpec:
22          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24          deterministic regardless of where the job ran."""
25
26          video_uri: str  # input (gs://? for cloud; a path/local:// for local)
27          out_uri: str  # output ROOT (gs://? bucket for cloud)
28          segmenter: Optional[str] = None
29          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30          config_overrides: tuple[str, ...] = ()
31
32
33      @dataclass(frozen=True)
34      class ComputeResult:
35          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40          returncode: int
41          outputs_uri: str = ""
42          video_id: str = ""
43          report: dict = field(default_factory=dict)
44          # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
45          # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
46          # executions describe <execution>). Empty for the in-process path / backends without
one.
47          execution: str = ""
```

```
48
49
50      class ComputeBackend(ABC):
51          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
52          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
53          execution identically."""
54
55          @abstractmethod
56          def run_infer(
57              self,
58              spec: ComputeJobSpec,
59              *,
60              on_wait: "Callable[[float], None] | None" = None,
61          ) -> ComputeResult:
62              """Run ``spec`` to completion and return its ``ComputeResult``.
63
64              ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
65              still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
66              long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
67
```

25. user (2026-07-07T15:05:11.607Z)

```
560          wasb_cfg = getattr(idx_cfg, "wasb", None)
561          if wasb_cfg is not None:
562              decode_backend = getattr(wasb_cfg, "decode_backend", None)
563              if decode_backend:
564                  overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565              batch_size = getattr(wasb_cfg, "batch_size", None)
566              if batch_size:
567                  overrides.append(f"indexing.wasb.batch_size={batch_size}")
568              prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569              if prefetch:
570                  overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571          spec = ComputeJobSpec(
572              video_uri=req.video_path,
573              out_uri=out_root,
574              segmenter=seg_name,
575              config_overrides=tuple(overrides),
576          )
577          try:
578              # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
579              # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
580              # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
581              # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
582              result = compute.run_infer(
583                  spec,
584                  on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
585              )
586          except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
587              logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
588              return _failed(f"cloud dispatch error: {e}", True)
589          if result.returncode != 0:
590              return _failed(f"cloud job failed (returncode {result.returncode}).", True)
591          if result.video_id and result.video_id != video_id:
592              # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
593              # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
```

```
diagnosable.
594             logger.warning(
595                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
596                 "and the worker hash; ingesting under the local id.",
597                 result.video_id,
598                 video_id,
599             )
600
601         notify("ingesting (cloud_run)")
602         storage_dict = (
603             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
604             if hasattr(storage_cfg, "model_dump")
```

26. assistant (2026-07-07T15:06:17.847Z)

Both callers are clear now. The worker `_dispatch_remote` catches only `PolicyTimeout` (?rc 4); the orchestration API path catches any `Exception` (? clean job failure). Let me check the `compute` package exports to decide where the new exception lives.

27. user (2026-07-07T15:06:19.376Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import dataclasses
13      import os
14      from typing import Any, Optional
15
16      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
17      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
18
19      __all__ = [
20          "ComputeBackend",
21          "ComputeJobSpec",
22          "ComputeResult",
23          "get_compute_backend",
24      ]
25
26
27      def _storage_dict(config: Any) -> dict:
28          storage = getattr(config, "storage", None)
29          if storage is None:
30              return {}
31          if hasattr(storage, "model_dump"):
32              return storage.model_dump()  # type: ignore[no-any-return]
33          return dict(storage) if isinstance(storage, dict) else {}
34
35
36      def get_compute_backend(
37          config: Any,
38          *,
```

```
        run_client: Optional[Any] = None,
        policy: ExecutionPolicy = DEFAULT_POLICY,
        token: Optional[str] = None,
        target_override: Optional[str] = None,
    ) -> Optional[ComputeBackend]:
        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
        in-process (the default-OFF path).

        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).

        ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
        per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
        server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
        """
        target = (
            target_override
            if target_override is not None
            else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
        )
        if target != "cloud_run":
            return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

        # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
        from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient

        # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
        # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
        # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild §4.3; the
old
        # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
        # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
        if policy is DEFAULT_POLICY:
            max_wait = getattr(
                getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
            )
            if max_wait:
                policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))

        job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
        region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
        project = os.environ.get("GOOGLE_CLOUD_PROJECT")
        client = run_client if run_client is not None else GoogleCloudRunJobClient()
        return CloudRunCompute(
            run_client=client,
            job_name=job_name,
            region=region,
            project=project,
            storage_config=_storage_dict(config),
            policy=policy,
            token=token,
        )
```

92

28. user (2026-07-07T15:06:20.194Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      from typing import Any, List
28
29      from backend import storage
30      from backend.compute import ComputeJobSpec, get_compute_backend
31      from backend.config import (
32          StartupCheckError,
33          enforce_startup_checks,
34          load_config,
35          probe_cuda_available,
36      )
37      from backend.infrastructure.database import Database
38      from backend.pipeline.segmenters.base import segmenter_registry
39      from backend.pipeline.validators.base import registry as validator_registry
40      from backend.reporting import emit_indexer_report
```

29. assistant (2026-07-07T15:07:10.304Z)

Now let me check the mypy config (to ensure my SDK-touching code passes type checks the way the existing google.cloud import does) and confirm how tests are run.

30. user (2026-07-07T15:07:15.407Z)

```
=== mypy.ini ===
[mypy]
```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
```

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

```
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set by create_app() / main()). Route handlers use request.app.state exclusively; helpers take explicit params. backend.api.job_worker lifted from quarantine (now takes explicit config/db params).

```
[mypy-backend.main]
ignore_errors = True
```

Worker dispatcher: composes the SAME validator/segmenter/report building blocks a backend.main, which are typed `config: dict` but receive the AppConfig migration shim at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine until the typed-config migration removes the dict?AppConfig duality everywhere.

```
[mypy-backend.worker.__main__]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here. (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True

[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
=== run_all_tests head ===
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
```

```
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
=== ruff/mypy/pytest versions ===
ruff 0.15.16
mypy 2.1.0 (compiled: yes)
pytest 9.0.3
3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.1944 64 bit (AMD64)]
```

31. assistant (2026-07-07T15:07:23.512Z)

```
`backend.compute.cloud_run` and `base`/`__init__` are type-checked (not quarantined);
`ignore_missing_imports=True` handles the `google.cloud` import. `backend.worker.__main__` is
quarantined. Let me check the ruff config (line length, ignores) before writing.
```

32. user (2026-07-07T15:07:26.261Z)

```
Exit code 2
=== ruff config ===
```

Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would bury blame history for zero behavior value).

```
#
```

Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle warnings (W) + bugbear (B) + isort (I ? consistent import ordering).

```
target-version = "py38"
```

scratch/ = throwaway debugging scripts that may hit real APIs; it is also excluded from pytest collection (see pytest.ini).

```
extend-exclude = ["scratch"]

[lint]
extend-select = ["W", "B", "I"]

[lint.per-file-ignores]
```

Entrypoint scripts deliberately run setup (load_dotenv, logging config, sys.path) before importing backend modules ? import order is load-bearing.

```
"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
"scripts/run_process_and_stitch.py" = ["E402"]
=== setup.cfg ruff? ===
```

33. assistant (2026-07-07T15:07:47.629Z)

No E501 enforcement (long lines are fine); I'll match the file's `# noqa: BLE001` convention on broad excepts. Now I have full context. Let me mark the chapter and implement.

Design summary:
- Add `RemoteExecutionFailed(RuntimeError)` to the compute contract (`base.py`), exported via `compute/__init__.py`.
- Add a concrete **default-`None`** `execution_terminal_state()` to `CloudRunJobClient` (backward-compatible; existing fakes unaffected), with a real Executions-API impl in `GoogleCloudRunJobClient`.
- In `run_infer`, replace the marker-only poll `check` with `_poll_check` that checks the marker first (authoritative success) and, when absent, consults execution status ? raising `RemoteExecutionFailed` fast on a terminal-without-marker (after a final marker re-check to close the write race).
- `_dispatch_remote` catches `RemoteExecutionFailed` ? distinct **rc 5**.

34. user (2026-07-07T15:07:48.390Z)

Chapter marked: "Harden cloud_run dispatch". Continue your current work.

35. assistant (2026-07-07T15:07:51.339Z)

Now the edits. First, the exception in the compute contract:

36. user (2026-07-07T15:07:58.175Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

37. user (2026-07-07T15:08:04.226Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

38. assistant (2026-07-07T15:08:06.849Z)

Now the core changes in `cloud_run.py`. First the import:

39. user (2026-07-07T15:08:08.889Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

40. assistant (2026-07-07T15:08:11.613Z)

Now add the `execution_terminal_state` method to the abstract client (concrete default-`None` for backward compatibility):

41. user (2026-07-07T15:08:20.214Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

42. assistant (2026-07-07T15:08:22.262Z)

Now the real implementation in `GoogleCloudRunJobClient` (after `cancel_execution`):

43. user (2026-07-07T15:08:32.405Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

44. assistant (2026-07-07T15:08:36.785Z)

Now wire the execution-status check into `run_infer`'s poll and add the helpers:

45. user (2026-07-07T15:08:42.952Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

46. assistant (2026-07-07T15:08:45.627Z)

Now add the `_poll_check` and `_terminal_execution_state` helpers, placed just before `_handle_poll_timeout`:

47. user (2026-07-07T15:09:00.604Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

48. assistant (2026-07-07T15:09:04.885Z)

Now update the module docstring (step 2) to document the new fast negative path:

49. user (2026-07-07T15:09:12.202Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

50. assistant (2026-07-07T15:09:17.967Z)

Now the caller ? `_dispatch_remote` in the worker: catch `RemoteExecutionFailed` ? distinct rc 5:

51. user (2026-07-07T15:09:26.602Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

52. assistant (2026-07-07T15:11:22.999Z)

Core changes are done. Now the tests. First, add the import and a new fake client:

53. user (2026-07-07T15:11:26.627Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

54. user (2026-07-07T15:11:36.882Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-07-07T15:11:41.844Z)

Now add the test functions after the poll-timeout section
(`test_poll_timeout_late_marker_rescues_result`):

56. user (2026-07-07T15:12:12.311Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

57. assistant (2026-07-07T15:12:32.519Z)

Let me run the targeted test file to validate the changes:

58. user (2026-07-07T15:12:35.583Z)

```
=================================== ERRORS ====================================
_________________ ERROR collecting tests/test_compute_backend.py __________________
ImportError while importing test module 'C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_compute_backend.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_compute_backend.py:15: in <module>
    from backend.compute import (
backend\compute\__init__.py:16: in <module>
    from backend.compute.base import (
E   ImportError: cannot import name 'ComputeJobSpec' from 'backend.compute.base' (C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\base.py)
=========================== short test summary info ===========================
ERROR tests/test_compute_backend.py
!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!
1 error in 0.47s
```

59. assistant (2026-07-07T15:12:38.999Z)

Import error ? let me check what happened to `base.py`:

60. user (2026-07-07T15:12:39.876Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Callable, Optional
18
19
20      class RemoteExecutionFailed(RuntimeError):
```

```
21          """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22          completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23          an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24          Succeeded run.
25
26          Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27          running): this says the execution is DEFINITIVELY over and no marker will ever
appear, so the
28          dispatcher must stop bucket-polling immediately instead of burning the full
29          ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30          job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31          the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
33          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
34          deterministic regardless of where the job ran."""
35
36          video_uri: str  # input (gs://? for cloud; a path/local:// for local)
37          out_uri: str  # output ROOT (gs://? bucket for cloud)
38          segmenter: Optional[str] = None
39          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
40          config_overrides: tuple[str, ...] = ()
41
42
43      @dataclass(frozen=True)
44      class ComputeResult:
45          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
```

61. assistant (2026-07-07T15:12:45.158Z)

My first edit accidentally dropped the `@dataclass`/`class ComputeJobSpec:` header. Let me restore it:

62. user (2026-07-07T15:12:49.808Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\compute\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

63. user (2026-07-07T15:13:03.516Z)

```
............................                                                [100%]
28 passed in 8.25s
```

64. assistant (2026-07-07T15:13:11.046Z)

All 28 pass. Now let me branch from `origin/master` (per the concurrent-session safety protocol) ? my uncommitted edits don't touch any file that differs between the current branch and master, so they'll carry over cleanly.

65. user (2026-07-07T15:13:15.579Z)

```
M       backend/compute/__init__.py
M       backend/compute/base.py
```

```
M       backend/compute/cloud_run.py
M       backend/worker/__main__.py
M       tests/test_compute_backend.py
branch 'fix/cloud-run-exec-status-fastfail' set up to track 'origin/master'.
Switched to a new branch 'fix/cloud-run-exec-status-fastfail'
---NOW ON---
fix/cloud-run-exec-status-fastfail
---HEAD (should be master's)---
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
---UNCOMMITTED (my edits carried over)---
 M backend/compute/__init__.py
 M backend/compute/base.py
 M backend/compute/cloud_run.py
 M backend/worker/__main__.py
 M tests/test_compute_backend.py
?? score_golden_gate.py
?? score_local.py
```

66. assistant (2026-07-07T15:13:22.156Z)

On the right branch. Now the pre-merge gate ? ruff, mypy, then the full suite with coverage. Let me start with the fast checks:

67. user (2026-07-07T15:13:31.528Z)

```
=== ruff ===
176 |         if not os.path.exists(path):
177 |             rows.append({"label": label, "ok": False, "error": "not found: " + path});
continue
    |                                                                                    ^
178 |         try:
179 |             rows.append(score_one(label, path, gts, mods, args.fps, args.frame_width))
    |

E702 Multiple statements on one line (semicolon)
   --> score_local.py:181:102
    |
179 |             rows.append(score_one(label, path, gts, mods, args.fps, args.frame_width))
180 |         except Exception as e:
181 |             rows.append({"label": label, "ok": False, "error": "{}:
{}".format(type(e).__name__, e)}); traceback.print_exc()
    |
^
182 |     print_table(rows, len(gts), args.name, mods["tm"])
183 |     return 0
    |

Found 18 errors.
[*] 4 fixable with the `--fix` option.
=== mypy (matches CI: backend training) ===
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 148 source files
```

68. assistant (2026-07-07T15:13:38.260Z)

mypy is clean. The ruff errors are all in the two untracked dev-tool scripts (`score_local.py`,

`score_golden_gate.py`), not my changes. Let me confirm by scoping ruff to the tracked tree / my files:

69. user (2026-07-07T15:13:42.478Z)

```
=== ruff on my edited files ===
All checks passed!
=== ruff on full tracked tree (exclude untracked dev scripts) ===
All checks passed!
=== confirm those 2 are untracked ===
?? score_golden_gate.py
?? score_local.py
```

70. assistant (2026-07-07T15:13:49.517Z)

My changes are clean for ruff and mypy. Now the full suite with coverage (floor 70), matching CI:

71. user (2026-07-07T15:18:51.200Z)

```
backend\pipeline\segmenters\base.py                    50      7    86%
backend\pipeline\segmenters\fusion_hybrid.py           87      1    99%
backend\pipeline\segmenters\gemini.py                 310     73    76%
backend\pipeline\segmenters\trajectory_hybrid.py      203      4    98%
backend\pipeline\segmenters\wasb_hybrid.py             23      2    91%
backend\pipeline\segmenters\yolo_hybrid.py            239     43    82%
backend\pipeline\stitcher\compiler.py                 239     51    79%
backend\pipeline\stitcher\watermark_i18n.py            23      1    96%
backend\pipeline\validators\base.py                    39      3    92%
backend\providers\__init__.py                          16      1    94%
backend\providers\base.py                               9      1    89%
backend\providers\gemini.py                           137     21    85%
backend\reporting\__init__.py                          26      2    92%
backend\reporting\harvest.py                          111      8    93%
backend\server_runtime.py                              57      2    96%
backend\sports\__init__.py                             17      1    94%
backend\sports\badminton.py                            13      1    92%
backend\sports\base.py                                 12      3    75%
backend\storage\__init__.py                            85      3    96%
backend\storage\base.py                                16      1    94%
backend\storage\capabilities.py                        59     10    83%
backend\storage\gcs.py                                 80      3    96%
backend\storage\gdrive.py                              61      3    95%
backend\storage\gdrive_api.py                         105      6    94%
backend\storage\local.py                               49      3    94%
backend\storage\s3.py                                  47      5    89%
backend\tools\export.py                                75      2    97%
backend\tools\rally_slicer.py                         105     47    55%
backend\utils\annotation_proxy.py                      64     11    83%
backend\utils\ffmpeg.py                                47      3    94%
backend\utils\freespace.py                             32      2    94%
backend\utils\paths.py                                 44      3    93%
backend\utils\run_telemetry.py                        126     12    90%
backend\utils\single_instance.py                       43     11    74%
backend\utils\video.py                                 34      3    91%
backend\worker\__main__.py                            315     29    91%
----------------------------------------------------------------------
TOTAL                                               12160   1758    86%

42 files skipped due to complete coverage.
Required test coverage of 70% reached. Total coverage: 85.54%
1790 passed, 1 skipped, 25 warnings in 293.30s (0:04:53)
======================================================================
 RESULT: PASS - all tests green
```

```
=============================================================================
```

72. assistant (2026-07-07T15:19:02.268Z)

Full gate is green: **1790 passed / 1 skipped**, coverage **85.54%** (floor 70), ruff clean on my files, mypy clean. Let me verify my new code in `cloud_run.py` is actually exercised (no uncovered non-pragma lines):

73. user (2026-07-07T15:19:17.731Z)

```
Name                            Stmts   Miss  Cover   Missing
backend\compute\base.py            21      0   100%
backend\compute\cloud_run.py      129     22    83%   124-143, 160-170, 179-190, 285-290, 331,
348, 373-378, 428-430
TOTAL                             150     22    85%
```

74. assistant (2026-07-07T15:19:33.263Z)

Let me map those missing lines to confirm they're all real-SDK (pragma) or pre-existing lines, not my new core logic:

75. user (2026-07-07T15:19:35.364Z)

```
70:    def run_job(
82:    def cancel_execution(self, execution: str) -> None:
88:    def execution_terminal_state(self, execution: str) -> Optional[str]:
104:class GoogleCloudRunJobClient(CloudRunJobClient):
108:    def run_job(
126:        except Exception as exc:  # pragma: no cover - real SDK not present in CI
133:        )  # pragma: no cover - exercised only on the real M7 run
149:    def cancel_execution(self, execution: str) -> None:
162:        except Exception as exc:  # pragma: no cover - real SDK not present in CI
169:        )  # pragma: no cover - exercised only on a real run
172:    def execution_terminal_state(self, execution: str) -> Optional[str]:
181:        except Exception:  # pragma: no cover - real SDK not present in CI
185:        )  # pragma: no cover - exercised only on a real run
186:        exe = client.get_execution(  # pragma: no cover - real run only
191:            return None  # pragma: no cover - real run only
193:        if getattr(exe, "cancelled_count", 0):  # pragma: no cover - real run only
195:        if getattr(exe, "failed_count", 0):  # pragma: no cover - real run only
197:        if getattr(exe, "succeeded_count", 0):  # pragma: no cover - real run only
199:        return "Failed"  # pragma: no cover - terminal but no positive success ? treat as
failed
202:class CloudRunCompute(ComputeBackend):
304:    def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
332:        raise RemoteExecutionFailed(
342:    def _terminal_execution_state(self, execution: str) -> Optional[str]:
359:    def _handle_poll_timeout(
```

76. assistant (2026-07-07T15:19:59.063Z)

Let me read my two new helper methods with line numbers to check the focused-run's uncovered lines (331, 348):

77. user (2026-07-07T15:20:00.434Z)

```
304         def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
305             """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
306
307             The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
308             wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
```

```
309             FAILURE* marker resolves through the normal path with its real returncode. Only
when the
310             marker is absent do we consult the execution status: a worker that TERMINATED
without ever
311             writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
312             otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
313             opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
314             failure.
315
316             Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
317             ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
318             will ever appear)."""
319             marker = self._read_marker(done_uri)
320             if marker is not None:
321                 return marker  # success signal ? authoritative even if status later flips
322             state = self._terminal_execution_state(execution)
323             if state is None:
324                 return None  # still pending/running, or status unknown ? keep bucket-
polling
325             # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
326             # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
327             # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
328             # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
329             marker = self._read_marker(done_uri)
330             if marker is not None:
331                 return marker
332             raise RemoteExecutionFailed(
333                 f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
334                 f"terminal state {state!r} with NO completion marker ? the worker exited
before "
335                 "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
336                 "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
337                 "inspect the Cloud Run Job execution log to diagnose: "
338                 f"`gcloud run jobs executions describe {execution or '<execution>'} "
339                 f"--region {self._region}`."
340             )
341
342         def _terminal_execution_state(self, execution: str) -> Optional[str]:
343             """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
344             flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
345             so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
346             ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
347             if not execution:
348                 return None
349             try:
350                 return self._client.execution_terminal_state(execution)
351             except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
352                 LOG.debug(
353                     "cloud-run: execution status probe failed for %s ? treating as unknown",
354                     execution,
```

```
355                     exc_info=True,
356                 )
357                 return None
358
359         def _handle_poll_timeout(
```

78. assistant (2026-07-07T15:20:35.859Z)

Two genuine branches in my new logic are uncovered ? line 331 (the race-guard where the marker lands between the two reads) and line 348 (empty execution name). Both are worth locking in. Let me add tests for them:

79. user (2026-07-07T15:20:48.429Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

80. user (2026-07-07T15:21:04.945Z)

```
Name                            Stmts   Miss  Cover   Missing
backend\compute\base.py            21      0   100%
backend\compute\cloud_run.py      129     20    84%   124-143, 160-170, 179-190, 285-290,
373-378, 428-430
TOTAL                             150     20    87%
============================== 30 passed in 8.86s ==============================
```

81. assistant (2026-07-07T15:21:30.393Z)

Lines 331 and 348 are now covered (30 passed). The remaining missing lines are the real-`GoogleCloudRunJobClient` SDK bodies (never run in CI by design). Let me confirm the coverage config's pragma handling and that my real-client guard line is genuinely exercised:

82. user (2026-07-07T15:21:32.747Z)

```
Exit code 2
=== coverage config ===
[run]
```

Excluded from coverage: the legacy motion segmenter (demo baseline with fabricated metadata) and the concrete validators (thin OpenCV/ffprobe wrappers only exercisable against real media files). validators/base.py stays counted ? it has a dedicated unit suite.

```
omit =
    backend/pipeline/segmenters/motion.py
    backend/pipeline/validators/format.py
    backend/pipeline/validators/resolution_fps.py
    backend/pipeline/validators/pts_continuity.py
    backend/pipeline/validators/scene_cut.py
    backend/pipeline/validators/blur.py
    backend/pipeline/validators/relevance.py
```

83. user (2026-07-07T15:21:42.185Z)

```
172         def execution_terminal_state(self, execution: str) -> Optional[str]:
173             # Advisory status probe (see the ABC docstring): describe the execution and,
ONLY once it
174             # is terminal (``completion_time`` set), classify it. A non-resolvable name /
absent SDK
175             # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this
never
176             # raises on bad input, because the probe is best-effort and must not fail a
```

```
healthy run.
177             if "/executions/" not in (execution or ""):
178                 return None
179             try:
180                 from google.cloud import run_v2  # type: ignore[attr-defined]
181             except Exception:  # pragma: no cover - real SDK not present in CI
182                 return None
183             client = (
184                 run_v2.ExecutionsClient()
185             )  # pragma: no cover - exercised only on a real run
186             exe = client.get_execution(  # pragma: no cover - real run only
187                 request=run_v2.GetExecutionRequest(name=execution)
188             )
189             # completion_time is a proto Timestamp: unset ? seconds==0 ? still
running/pending.
190             if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
191                 return None  # pragma: no cover - real run only
192             # Terminal: prefer the explicit per-task counts (a cancel shows as
cancelled_count>0).
193             if getattr(exe, "cancelled_count", 0):  # pragma: no cover - real run only
194                 return "Cancelled"
195             if getattr(exe, "failed_count", 0):  # pragma: no cover - real run only
196                 return "Failed"
197             if getattr(exe, "succeeded_count", 0):  # pragma: no cover - real run only
198                 return "Succeeded"
199             return "Failed"  # pragma: no cover - terminal but no positive success ? treat
as failed
```

84. assistant (2026-07-07T15:22:20.392Z)

The guard (177-178) is covered; the remaining uncovered lines (179-180, 190) are real-SDK-only lines interleaved with pragmas ? the exact same coverage profile as the pre-existing `run_job`/`cancel_execution` real-client bodies (documented as never exercised in CI). My testable `CloudRunCompute` logic is fully covered. Let me review the full diff for coherence:

85. user (2026-07-07T15:22:22.048Z)

```
 backend/compute/__init__.py   |   8 +-
 backend/compute/base.py       |  14 +++
 backend/compute/cloud_run.py  | 115 ++++++++++++++++++++-
 backend/worker/__main__.py    |  16 ++-
 tests/test_compute_backend.py | 229 ++++++++++++++++++++++++++++++++++++++++++-
 5 files changed, 375 insertions(+), 7 deletions(-)
====================== FULL DIFF (source only) ======================
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
index 1091582..482f3c9 100644
--- a/backend/compute/__init__.py
+++ b/backend/compute/__init__.py
@@ -13,13 +13,19 @@ import dataclasses
 import os
 from typing import Any, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)
 from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

 __all__ = [
     "ComputeBackend",
     "ComputeJobSpec",
     "ComputeResult",
```

```
+    "RemoteExecutionFailed",
     "get_compute_backend",
 ]

diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..8f34547 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -17,6 +17,20 @@ from dataclasses import dataclass, field
 from typing import Callable, Optional


+class RemoteExecutionFailed(RuntimeError):
+    """A dispatched remote job's execution reached a TERMINAL state without ever writing its
+    completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image argparse-
rejecting
+    an unknown flag) before the marker push, or the marker upload itself failed on an otherwise
+    Succeeded run.
+
+    Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL be
+    running): this says the execution is DEFINITIVELY over and no marker will ever appear, so
the
+    dispatcher must stop bucket-polling immediately instead of burning the full
+    ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message carries the
+    job + execution name + terminal state + the ``gcloud`` command to read the execution log,
so
+    the whole "worker crashed before the done-marker" class fails fast AND diagnostically."""
+
+
 @dataclass(frozen=True)
 class ComputeJobSpec:
     """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7e0c3cc 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -11,6 +11,10 @@ Flow (the cloud-serving compute path):
      job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
healthy
      worker that finished just past the deadline is rescued, not wasted ? then best-effort
CANCELS
      the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
+     *Fast negative path:* while the marker stays the SOLE success signal, each poll also
probes the
+     Cloud Run Executions API ? if the execution has TERMINATED without ever writing a marker
(a
+     worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
+     (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile wait.
   3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
      ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -32,7 +36,12 @@ import uuid
 from abc import ABC, abstractmethod
 from typing import Any, Callable, List, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)
 from backend.infrastructure.execution_policy import (
     DEFAULT_POLICY,
```

```
     ExecutionPolicy,
@@ -76,6 +85,21 @@ class CloudRunJobClient(ABC):
         May raise ? the caller treats every failure as best-effort (the original poll timeout
is
         NEVER masked by a cancel error)."""

+    def execution_terminal_state(self, execution: str) -> Optional[str]:
+        """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
+        ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or the state
+        can't be determined).
+
+        ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a worker
that
+        terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails fast
instead
+        of bucket-polling the full budget. The done-marker stays the ONLY success signal.
+
+        NON-abstract with a default of ``None`` (additive / default-OFF): a client that can't ?
or
+        chooses not to ? poll the Executions API degrades transparently to today's marker-only
+        polling. Implementations MAY raise on a transient API error ? ``run_infer`` treats a
raise
+        as ``None`` (unknown ? keep waiting), so a flaky status API never fails a healthy
run."""
+        return None
+

 class GoogleCloudRunJobClient(CloudRunJobClient):
     """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI
uses
@@ -145,6 +169,35 @@ class GoogleCloudRunJobClient(CloudRunJobClient):
         )  # pragma: no cover - exercised only on a real run
         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

+    def execution_terminal_state(self, execution: str) -> Optional[str]:
+        # Advisory status probe (see the ABC docstring): describe the execution and, ONLY once
it
+        # is terminal (``completion_time`` set), classify it. A non-resolvable name / absent
SDK
+        # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
+        # raises on bad input, because the probe is best-effort and must not fail a healthy
run.
+        if "/executions/" not in (execution or ""):
+            return None
+        try:
+            from google.cloud import run_v2  # type: ignore[attr-defined]
+        except Exception:  # pragma: no cover - real SDK not present in CI
+            return None
+        client = (
+            run_v2.ExecutionsClient()
+        )  # pragma: no cover - exercised only on a real run
+        exe = client.get_execution(  # pragma: no cover - real run only
+            request=run_v2.GetExecutionRequest(name=execution)
+        )
+        # completion_time is a proto Timestamp: unset ? seconds==0 ? still running/pending.
+        if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
+            return None  # pragma: no cover - real run only
+        # Terminal: prefer the explicit per-task counts (a cancel shows as cancelled_count>0).
+        if getattr(exe, "cancelled_count", 0):  # pragma: no cover - real run only
+            return "Cancelled"
+        if getattr(exe, "failed_count", 0):  # pragma: no cover - real run only
+            return "Failed"
+        if getattr(exe, "succeeded_count", 0):  # pragma: no cover - real run only
+            return "Succeeded"
```

```
+        return "Failed"  # pragma: no cover - terminal but no positive success ? treat as
failed
+

 class CloudRunCompute(ComputeBackend):
     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
@@ -211,7 +264,7 @@ class CloudRunCompute(ComputeBackend):

         try:
             marker = poll_until(
-                lambda: self._read_marker(done_uri),
+                lambda: self._poll_check(done_uri, str(execution or "")),
                 self._policy,
                 label="cloud-run-infer",
                 logger=LOG,
@@ -220,6 +273,9 @@ class CloudRunCompute(ComputeBackend):
                 on_tick=on_wait,
             )
         except PolicyTimeout as timeout_exc:
+            # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
+            # so it propagates past here uncaught ? the dispatch handler maps it to a distinct,
+            # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
         video_id = str(marker.get("video_id", ""))
         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
@@ -245,6 +301,61 @@ class CloudRunCompute(ComputeBackend):
             execution=str(execution or ""),
         )

+    def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
+        """One bucket-poll iteration, hardened with an Executions-API fast negative path.
+
+        The done-marker is the SOLE success signal and is checked FIRST ? a present marker
always
+        wins (even if the execution's status later flips), so a worker that wrote a SUCCESS *or
a
+        FAILURE* marker resolves through the normal path with its real returncode. Only when
the
+        marker is absent do we consult the execution status: a worker that TERMINATED without
ever
+        writing a marker (crash / OOM / an old image argparse-aborting on an unknown flag)
would
+        otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min) before
failing
+        opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
+        failure.
+
+        Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
+        ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over, no
marker
+        will ever appear)."""
+        marker = self._read_marker(done_uri)
+        if marker is not None:
+            return marker  # success signal ? authoritative even if status later flips
+        state = self._terminal_execution_state(execution)
+        if state is None:
+            return None  # still pending/running, or status unknown ? keep bucket-polling
+        # The execution is terminal but no marker is present. Re-check the marker ONCE to close
the
+        # race where the worker wrote it between our two reads (mirrors the post-timeout rescue
in
```

```
+        # _handle_poll_timeout). If it is genuinely absent, the process is gone and no marker
will
+        # EVER appear ? fail fast + diagnostically rather than wait out the whole poll budget.
+        marker = self._read_marker(done_uri)
+        if marker is not None:
+            return marker
+        raise RemoteExecutionFailed(
+            f"cloud-run: job={self._job_name} execution={execution or '<unknown>'} reached "
+            f"terminal state {state!r} with NO completion marker ? the worker exited before "
+            "writing one (crash / OOM / pre-marker abort, or a failed marker upload on an "
+            "otherwise-Succeeded run). The execution is over, so the marker will never appear;
"
+            "inspect the Cloud Run Job execution log to diagnose: "
+            f"`gcloud run jobs executions describe {execution or '<execution>'} "
+            f"--region {self._region}`."
+        )
+
+    def _terminal_execution_state(self, execution: str) -> Optional[str]:
+        """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A raising
or
+        flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
+        so any exception ? and an empty/unresolvable execution name ? degrades to ``None``
(unknown
+        ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
+        if not execution:
+            return None
+        try:
+            return self._client.execution_terminal_state(execution)
+        except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the bucket-poll
+            LOG.debug(
+                "cloud-run: execution status probe failed for %s ? treating as unknown",
+                execution,
+                exc_info=True,
+            )
+            return None
+
     def _handle_poll_timeout(
         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
     ) -> dict:
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..aefb73d 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -176,9 +176,14 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
     container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.

-    A remote job that never writes a completion marker (a failed/hung container) makes the
bucket
-    poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
dispatch
-    timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
+    A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
+    from the local 2/3), never a raw traceback:
+      * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution may
still be
+        running (the shim best-effort cancels it).
+      * rc 5 ? the execution has TERMINATED without a marker (``RemoteExecutionFailed``): a
worker
+        crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of the
```

```
+        ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
+    from backend.compute import RemoteExecutionFailed
     from backend.infrastructure.execution_policy import PolicyTimeout

     spec = ComputeJobSpec(
@@ -189,6 +194,11 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
     )
     try:
         result = compute.run_infer(spec)
+    except RemoteExecutionFailed as e:
+        logger.error(
+            "[worker] remote execution terminated without a completion marker: %s", e
+        )
+        return 5
     except PolicyTimeout as e:
         logger.warning(
             "[worker] remote compute did not complete (no marker within budget): %s", e
```

86. assistant (2026-07-07T15:22:39.457Z)

The diff is coherent. Let me run the full suite one final time to certify the exact tree I'm about to commit (I added 2 tests since the last full run), with the coverage gate:

87. user (2026-07-07T15:27:12.142Z)

```
backend\utils\single_instance.py                      43     11    74%
backend\utils\video.py                                34      3    91%
backend\worker\__main__.py                           315     29    91%
----------------------------------------------------------------------
TOTAL                                              12160   1756    86%

42 files skipped due to complete coverage.
Required test coverage of 70% reached. Total coverage: 85.56%
1792 passed, 1 skipped, 25 warnings in 266.06s (0:04:26)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

88. assistant (2026-07-07T15:27:33.054Z)

Final gate certified: **1792 passed / 1 skipped**, coverage **85.56%** (floor 70, no regression), ruff + mypy clean. Let me commit on the feature branch ? re-verifying branch/HEAD and staging explicit paths only (never the untracked dev scripts), per the concurrent-session safety protocol:

89. user (2026-07-07T15:27:35.816Z)

```
=== branch + HEAD (mine) ===
fix/cloud-run-exec-status-fastfail
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
=== stage explicit paths only ===
=== staged ===
 backend/compute/__init__.py   |   8 +-
 backend/compute/base.py       |  14 +++
 backend/compute/cloud_run.py  | 115 ++++++++++++++++++++++-
 backend/worker/__main__.py    |  16 ++-
 tests/test_compute_backend.py | 229 ++++++++++++++++++++++++++++++++++++++++++-
 5 files changed, 375 insertions(+), 7 deletions(-)
=== untracked NOT staged (dev tools) ===
M  backend/compute/__init__.py
M  backend/compute/base.py
M  backend/compute/cloud_run.py
M  backend/worker/__main__.py
```

```
M  tests/test_compute_backend.py
?? score_golden_gate.py
?? score_local.py
```

90. assistant (2026-07-07T15:27:39.915Z)

Only my 5 files staged; the two dev scripts remain unstaged. HEAD is still `origin/master` (no sibling commit slipped in). Committing:

91. user (2026-07-07T15:27:52.262Z)

```
[fix/cloud-run-exec-status-fastfail 4e04c97] fix(cloud-run): fail fast when a worker execution terminates without a done-marker
 5 files changed, 375 insertions(+), 7 deletions(-)
=== verify: only my commit vs origin/master ===
4e04c97 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
```

92. assistant (2026-07-07T15:28:05.884Z)

Committed on the feature branch ? `origin/master..HEAD` lists **only** my commit; no sibling commit rode in. Per the established pattern of tracking local-unpushed branches in memory (like the compute-local and validation-decode branches), let me record this one so it isn't lost:

93. assistant (2026-07-07T15:28:18.316Z)

```
---
name: cloud-run-exec-status-fastfail
description: "Local UNPUSHED branch fix/cloud-run-exec-status-fastfail ? run_infer now polls the Executions API to fail fast when a worker crashes before the done-marker"
metadata:
  type: project
---

**Branch `fix/cloud-run-exec-status-fastfail`** (commit `4e04c97`, based on `origin/master` @ 3be0935) ? **local, gated, NOT pushed** (like [[compute-local-hosted-guardrail]] / [[validation-shared-decode-pass]]). Full gate reproduced locally: full suite **1792 passed /1 skipped**, coverage **85.56%** (floor 70, no regression), `ruff` + `mypy backend training` clean.

**What it fixes:** `CloudRunCompute.run_infer` (backend/compute/cloud_run.py) bucket-polled the done-marker ONLY; a worker that terminated before writing the marker (old image argparse-aborting
on an unknown flag like `--skip-validation`, OOM, any pre-marker exception) made the dispatcher poll the full `deployment.remote_poll_max_wait_sec` (~65 min) then fail opaquely.

**How:** each poll iteration (`_poll_check`) checks the marker FIRST (sole success signal ? a worker SUCCESS *or FAILURE* marker still wins), and when it's absent probes the Cloud Run Executions API via the new NON-abstract `CloudRunJobClient.execution_terminal_state` (default `None` = additive/default-OFF; real impl uses `get_execution` completion_time + per-task counts).
A terminal execution + no marker ? `RemoteExecutionFailed` (new, in `backend/compute/base.py`) after ONE final marker re-check to close the write race. `_dispatch_remote` maps it to **rc 5** (distinct from rc 4 poll-timeout, which means the exec may still be running). Probe is best-effort:
a raising/flaky status API is swallowed ? keep polling, never fails a healthy run.

**Why it exists:** surfaced by the adversarial review of the worker-skip-revalidation PR (`fix/worker-skip-revalidation-on-dispatch`), whose interim mitigation was a "deploy control-plane
+ worker together" note. This is the durable fix for the whole "worker crashed before the done-marker" class.

**Next:** owner to push + open PR (I did not push/PR ? not asked). Real-client `execution_terminal_state`
```

SDK body is `# pragma: no cover` (never run in CI, same convention as
`run_job`/`cancel_execution`).

94. user (2026-07-07T15:28:18.379Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\cloud-run-exec-status-fastfail.md (file state is current in your context ? no need to Read it back)

95. user (2026-07-07T15:28:21.013Z)

1 # Memory index
2
3 - ? **Start here (GPU-resident rollout):** session-handoff-gpu-resident.md ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip golden gate.
4
5 - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under `khelsutra-guru/badminton-highlight-indexer`.
6 - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`); diagnose-first, run long work detached.
7 - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot, torch2.x/pynvc; what #501 should encode.
8 - [LGTM only when merge-ready](lgtm-only-when-merge-ready.md) ? post LGTM only when genuinely satisfied a PR is merge-ready (verified), for all PRs.
9 - [GPU-resident modernization status](gpu-resident-modernization.md) ? #506 validated clean; the resize must match cv2.warpAffine via grid_sample (not F.interpolate).
10 - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? PRs #511 (backend) + #169 (SPA) fixing the "weights not found" trap, gated + pushed. ? #511 touches orchestration.py + test_api_cloud_dispatch.py ? OVERLAPS PRs #512/#513 (merge-conflict; whoever lands 2nd rebases).
11 - [Validation shared decode pass](validation-shared-decode-pass.md) ? validators now share ONE decode pass (branch `perf/validation-shared-decode-pass`, d53851b, **PR #512** open); pure grab()-through is SLOWER than seeks ? use forward-hybrid.
12 - [PR #513 worker skip-revalidation](pr513-worker-skip-revalidation.md) ? worker skips redundant re-validation on cloud dispatch (fixes the blur-threshold CUJ). **MERGED** 2026-07-07 (squash 23c1868). ? landed first on orchestration.py + test_api_cloud_dispatch.py, so PRs #511/#512 must now REBASE onto master. Follow-up (crash-before-marker fast-fail) now running on branch `fix/cloud-run-exec-status-fastfail`.
13

96. assistant (2026-07-07T15:28:27.308Z)

The index already anticipates this exact follow-up (the pr513 entry names my branch). Let me add the index pointer for the new memory:

97. user (2026-07-07T15:28:33.013Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

98. assistant (2026-07-07T15:28:49.059Z)

The change is complete and verified. Here's a summary.

What was wrong

`CloudRunCompute.run_infer` waited for job completion by bucket-polling the done-marker **only** ? it never consulted the Cloud Run execution status. When a worker container terminated before writing the marker (an old image argparse-aborting on `--skip-validation`, an OOM, any pre-marker exception), the dispatcher polled the full `remote_poll_max_wait_sec` (~65 min) and then failed opaquely as a generic dispatch error.

The fix

Fast negative path in the bucket-poll ([cloud_run.py](backend/compute/cloud_run.py)):
- New `_poll_check` checks the **marker first** (still the sole success signal ? a worker-written SUCCESS *or FAILURE* marker resolves normally with its real rc). Only when the marker is absent does it probe the Executions API.
- A **terminal execution + no marker** raises the new `RemoteExecutionFailed` ? after **one final marker re-check** to close the write race (mirroring the existing post-timeout rescue). The diagnostic names the job, execution, terminal state, and the exact `gcloud ... executions describe` command.
- `execution_terminal_state` is added to `CloudRunJobClient` as a **non-abstract, default-`None`** method (additive/default-OFF: existing fakes and any client that can't poll status degrade transparently to today's marker-only behavior), with a real Executions-API impl in `GoogleCloudRunJobClient`.
- The probe is **best-effort** ? a raising/flaky status API is swallowed (keep polling), so it never fails a healthy run.
- [`_dispatch_remote`](backend/worker/__main__.py:174) maps `RemoteExecutionFailed` to a **distinct rc 5** (vs rc 4 poll-timeout, which means the execution may still be running). The orchestration API path already surfaces it fast + labelled via its `except Exception`.

Tests ([test_compute_backend.py](tests/test_compute_backend.py))

The requested test (fake execution reports **Failed**, no marker ? fast `RemoteExecutionFailed` on poll 0, **rc 5 not 4**), plus: the Succeeded-without-marker generalization, the marker-wins-over-terminal-status race, best-effort probe-error swallow ? falls back to `PolicyTimeout`, and the default/real-client `None` fallbacks. Bounded `max_wait_sec=2.0` so a regression fails in ~2s rather than hanging.

Gate (pre-merge, reproduced locally)

- **Full suite: 1792 passed / 1 skipped** (`python run_all_tests.py`)
- **Coverage: 85.56%** (floor 70, no regression; my testable logic fully covered ? only the real-SDK client body is uncovered, same `# pragma: no cover` convention as the existing `run_job`/`cancel_execution`)
- **ruff clean** on all changed files; **mypy `backend training` clean** (the only ruff failures are in two pre-existing *untracked* dev scripts, not part of this change and not staged)

Committed as `4e04c97` on branch **`fix/cloud-run-exec-status-fastfail`** (based on `origin/master`; `origin/master..HEAD` contains only this commit). I did **not** push or open a PR ? say the word and I'll push it and open one against master.

99. user (2026-07-07T15:28:51.526Z)

any update?

100. assistant (2026-07-07T15:28:58.679Z)

Let me ground the update in the current repo state rather than just recap from memory.

101. user (2026-07-07T15:29:01.103Z)

```
Exit code 128
fatal: not a git repository (or any of the parent directories): .git
=== branch ===
fatal: not a git repository (or any of the parent directories): .git
```

102. assistant (2026-07-07T15:29:05.981Z)

Working directory reset to the stub ? let me point back at the real repo:

103. user (2026-07-07T15:29:10.949Z)

```
=== branch ===
```

```
fix/cloud-run-exec-status-fastfail
=== my commit vs origin/master ===
4e04c97 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
=== origin/master HEAD ===
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
=== working tree (tracked) ===
=== unpushed vs upstream ===
4e04c97 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
```

104. assistant (2026-07-07T15:29:41.560Z)

origin/master **moved** ? it's now at `23c1868` (the #513 worker skip-revalidation merge), but my branch is still based on the older `3be0935`. That's exactly the concurrent-slates situation my conventions warn about. Let me assess whether my base is now stale and whether #513 overlaps my files.

105. user (2026-07-07T15:29:46.233Z)

```
=== is my base (3be0935) an ancestor of new origin/master (23c1868)? ===
YES ? my branch just needs rebasing onto 23c1868
=== files origin/master changed between my base and now ===
 backend/compute/base.py          |  7 +++++++
 backend/compute/cloud_run.py     |  4 ++++
 backend/orchestration.py         |  7 +++++++
 backend/worker/__main__.py       | 32 ++++++++++++++++++++++++++----
 deploy/cloudrun/README.md        | 12 ++++++++++++
 tests/test_api_cloud_dispatch.py | 16 ++++++++++++++++
 tests/test_compute_backend.py    | 24 ++++++++++++++++++++++++
 tests/test_worker.py             | 45 +++++++++++++++++++++++++++++++++++++++++++++
 8 files changed, 143 insertions(+), 4 deletions(-)
=== MY files ===
backend/compute/__init__.py
backend/compute/base.py
backend/compute/cloud_run.py
backend/worker/__main__.py
tests/test_compute_backend.py
=== OVERLAP (files touched by BOTH) ===
backend/compute/base.py
backend/compute/cloud_run.py
backend/worker/__main__.py
tests/test_compute_backend.py
```

106. assistant (2026-07-07T15:30:13.308Z)

#513 landed in `origin/master` **after** I branched, and it touched **4 of my 5 files** ? including `cloud_run.py`, `base.py`, and `worker/__main__.py`. I built my change on the pre-#513 tree, so I need to integrate with #513's now-merged work, not just textually rebase over it. Let me scout exactly what #513 changed in the overlapping files first.

107. user (2026-07-07T15:30:14.763Z)

```
################ #513 (23c1868) changes to backend/compute/base.py ################
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
```

```
and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
plane)
+    # validating exactly as before.
+    skip_validation: bool = False


 @dataclass(frozen=True)
############### #513 (23c1868) changes to backend/compute/cloud_run.py ###############
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
         ]
         if spec.segmenter:
             argv += ["--segmenter", spec.segmenter]
+        if spec.skip_validation:
+            # The control-plane already validated + only dispatches on PASS ? tell the worker
not to
+            # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
+            argv.append("--skip-validation")
         for kv in (*spec.config_overrides, *self._container_overrides):
             argv += ["--set", kv]

############### #513 (23c1868) changes to backend/worker/__main__.py ###############
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
         out_uri=args.out_uri,
         segmenter=args.segmenter,
         config_overrides=tuple(args.set or ()),
+        skip_validation=bool(getattr(args, "skip_validation", False)),
     )
     try:
         result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
     video_id = compute_video_id(local_video)

-    # 3. VALIDATE (same registry as the main CLI).
-    val_results, val_context = validator_registry.run_all_with_context(
-        local_video, config
-    )
+    # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+    # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+    # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+    # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+    # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+    # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+    # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
```

```
it; the
+    # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+    if getattr(args, "skip_validation", False):
+        print(
+            "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+            "gate and already validated this input."
+        )
+        val_results, val_context = [], {}
+    else:
+        val_results, val_context = validator_registry.run_all_with_context(
+            local_video, config
+        )
     failures = [r for r in val_results if not r.passed]
     db = Database(os.path.join(scratch, config.output.db_name))
     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
         metavar="k=v",
         help="config override, e.g. --set indexing.skip_ai_handoff=true",
     )
+    pi.add_argument(
+        "--skip-validation",
+        action="store_true",
+        help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+        "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+        "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+        "`worker infer` CLI still validates.",
+    )
     pi.set_defaults(func=cmd_infer)

     pt = sub.add_parser(
```

108. assistant (2026-07-07T15:31:18.724Z)

Now I fully understand the integration. #513 added an orthogonal `skip_validation` feature ? and critically, it's the *exact* scenario my fix guards against: a post-#513 control-plane dispatching `--skip-validation` to an **old** worker image would argparse-abort before the marker ? the ~65-min hang my change kills. The two compose cleanly; I need to rebase onto `23c1868` and reconcile. Let me rebase (my own unpushed commit ? safe):

109. user (2026-07-07T15:31:20.651Z)

```
Rebasing (1/1)

Successfully rebased and updated refs/heads/fix/cloud-run-exec-status-fastfail.
=== STATUS ===
?? score_golden_gate.py
?? score_local.py
```

110. assistant (2026-07-07T15:31:37.672Z)

The rebase applied cleanly ? git's 3-way merge placed #513's and my hunks without conflict (they were non-overlapping). But "clean rebase" ? "semantically correct." Let me verify both features survived intact in each overlapping file, then re-run the full gate on the new base.

111. user (2026-07-07T15:31:38.796Z)

```
=== base.py: BOTH RemoteExecutionFailed AND skip_validation present? ===
20:class RemoteExecutionFailed(RuntimeError):
51:    skip_validation: bool = False
=== cloud_run.py: BOTH --skip-validation argv AND my fast-fail present? ===
17:     (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
```

```
wait.
43:    RemoteExecutionFailed,
88:    def execution_terminal_state(self, execution: str) -> Optional[str]:
172:    def execution_terminal_state(self, execution: str) -> Optional[str]:
252:        if spec.skip_validation:
255:            argv.append("--skip-validation")
271:                lambda: self._poll_check(done_uri, str(execution or "")),
280:            # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
308:    def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
321:        ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over, no
marker
326:        state = self._terminal_execution_state(execution)
336:        raise RemoteExecutionFailed(
346:    def _terminal_execution_state(self, execution: str) -> Optional[str]:
347:        """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
354:            return self._client.execution_terminal_state(execution)
=== worker __main__.py _dispatch_remote: BOTH skip_validation= AND except RemoteExecutionFailed?
===
        crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of the
        ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
    from backend.compute import RemoteExecutionFailed
    from backend.infrastructure.execution_policy import PolicyTimeout

    spec = ComputeJobSpec(
        video_uri=args.video_uri,
        out_uri=args.out_uri,
        segmenter=args.segmenter,
        config_overrides=tuple(args.set or ()),
        skip_validation=bool(getattr(args, "skip_validation", False)),
    )
    try:
        result = compute.run_infer(spec)
    except RemoteExecutionFailed as e:
        logger.error(
            "[worker] remote execution terminated without a completion marker: %s", e
        )
        return 5
    except PolicyTimeout as e:
        logger.warning(
            "[worker] remote compute did not complete (no marker within budget): %s", e
```

112. assistant (2026-07-07T15:31:53.914Z)

All three source files integrate both features cleanly ? orthogonal and composed correctly. Now the empirical ground truth: re-run the full gate on the rebased (post-#513) base.

113. user (2026-07-07T15:36:39.318Z)

```
=== ruff (my files) ===
All checks passed!
=== mypy ===
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 148 source files
=== FULL SUITE + COVERAGE (floor 70) ===
TOTAL                                              12167   1756    86%

42 files skipped due to complete coverage.
Required test coverage of 70% reached. Total coverage: 85.57%
1796 passed, 1 skipped, 25 warnings in 274.38s (0:04:34)
======================================================================
```

```
 RESULT: PASS - all tests green
==========================================================================
```

114. assistant (2026-07-07T15:37:19.147Z)

Full gate is green on the rebased base: **1796 passed / 1 skipped**, coverage **85.57%**, ruff + mypy clean. Both features coexist correctly. Let me confirm the final branch state, then ? since this was a non-trivial integration (rebased across a sibling merge touching 4 of my 5 files) ? run an adversarial verification workflow to make sure the rebase didn't silently drop or subtly break either feature.

115. user (2026-07-07T15:37:21.896Z)

```
=== branch now based on 23c1868, only my commit ===
e99062a fix(cloud-run): fail fast when a worker execution terminates without a done-marker
=== my HEAD parent should be origin/master (23c1868) ===
e99062a fix(cloud-run): fail fast when a worker execution terminates without a done-marker
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
=== diff stat of my commit on new base ===
 backend/compute/__init__.py                  |   8 +-
 backend/compute/base.py                      |  14 +
 backend/compute/cloud_run.py                 | 115 ++++++-
 backend/pipeline/validators/base.py          |  98 ------
 backend/pipeline/validators/blur.py          |  56 ++--
 backend/pipeline/validators/frame_sampler.py | 143 ---------
 backend/pipeline/validators/relevance.py     |  93 +++---
 backend/pipeline/validators/scene_cut.py     |  99 +++---
 backend/worker/__main__.py                   |  16 +-
 docs/CODE_MAP.md                             |   9 +-
 tests/test_compute_backend.py                | 229 ++++++++++++-
 tests/test_frame_sampler.py                  | 444 ---------------------------
 12 files changed, 505 insertions(+), 819 deletions(-)
```

116. assistant (2026-07-07T15:38:20.826Z)

?? **Red flag.** `origin/master..HEAD` shows my commit "deleting" 7 files I never touched ? the entire `perf/validation-shared-decode-pass` set (`frame_sampler.py` -143, `test_frame_sampler.py` -444, `validators/*.py`). That's the "silently drop a sibling's work" hazard. It means my rebase base (23c1868) is **not** the current `origin/master` tree ? master has advanced. Let me diagnose precisely before doing anything else.

117. user (2026-07-07T15:38:32.086Z)

```
=== origin/master current SHA + subject ===
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
=== does CURRENT origin/master contain the validation work? ===
YES origin/master HAS test_frame_sampler.py
=== does my rebase base 23c1868 contain it? ===
NO ? 23c1868 predates the validation merge
=== my HEAD parent ===
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
=== merge-base(HEAD, origin/master) ===
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
=== is 23c1868 an ancestor of current origin/master? ===
YES
=== recent origin/master history ===
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag
(#507)
```

118. assistant (2026-07-07T15:38:57.441Z)

Now it's clear and **benign**. History is linear: `3be0935 ? 23c1868 (#513) ? 20862d7 (#512, current tip)`. My earlier fetch caught master at 23c1868; #512 (validation) landed on top *after* I rebased. My branch isn't diverged ? it's simply **one commit behind** current master (merge-base = 23c1868, an ancestor of the tip). The scary "deletions" are a two-**dot** diff artifact (comparing trees); a real merge uses the merge-base and my commit never touches those files. Let me prove that with the three-dot diff, then rebase forward onto the current tip (#512 is file-disjoint, so it'll be clean):

119. user (2026-07-07T15:39:03.671Z)

```
=== THREE-dot diff (what my branch ACTUALLY changes vs master's merge-base) ? should be ONLY my
5 files ===
 backend/compute/__init__.py   |   8 +-
 backend/compute/base.py       |  14 +++
 backend/compute/cloud_run.py  | 115 ++++++++++++++++++++++-
 backend/worker/__main__.py    |  16 ++-
 tests/test_compute_backend.py | 229 ++++++++++++++++++++++++++++++++++++++++++-
 5 files changed, 375 insertions(+), 7 deletions(-)

=== rebase forward onto current origin/master (20862d7) ===
Rebasing (1/1)

Successfully rebased and updated refs/heads/fix/cloud-run-exec-status-fastfail.
=== after: only my commit, now atop 20862d7 ===
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
--- parent ---
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
=== sanity: validation work still present in my branch now? ===
YES ? validation work intact in my branch
=== two-dot stat now (should ALSO be only my 5 files) ===
 backend/compute/__init__.py   |   8 +-
 backend/compute/base.py       |  14 +++
 backend/compute/cloud_run.py  | 115 ++++++++++++++++++++++-
 backend/worker/__main__.py    |  16 ++-
 tests/test_compute_backend.py | 229 ++++++++++++++++++++++++++++++++++++++++++-
 5 files changed, 375 insertions(+), 7 deletions(-)
```

120. assistant (2026-07-07T15:39:21.531Z)

Fully resolved ? the alarm was a two-dot-diff artifact from being one commit behind. Now rebased onto the current tip (20862d7), my commit (`145854e`) is the only change vs master, the three-dot and two-dot diffs both show exactly my 5 files, and the validation work is intact in my branch. Since the base changed again (added #512's validator refactor), let me re-certify the full gate on this final tree:

121. user (2026-07-07T15:46:58.984Z)

```
=== ruff (my files) ===
All checks passed!
=== mypy ===
backend\pipeline\validators\base.py:114: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 149 source files
=== FULL SUITE + COVERAGE (floor 70) on final base 20862d7+mine ===

42 files skipped due to complete coverage.
Required test coverage of 70% reached. Total coverage: 85.66%
1833 passed, 1 skipped, 25 warnings in 435.95s (0:07:15)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

122. assistant (2026-07-07T15:47:31.170Z)

Final gate is green on the fully-current base: **1833 passed / 1 skipped**, coverage **85.66%**, ruff + mypy clean. The change is empirically solid. Now, since this went through a non-trivial rebase across two sibling merges (#512 + #513), let me run an adversarial verification workflow ? independent reviewers checking that the integration preserved both features and that my fast-fail logic is correct, each finding then adversarially verified before I act on it.

123. user (2026-07-07T15:48:13.111Z)

Workflow launched in background. Task ID: wah0pctzi
Summary: Adversarially verify the rebased cloud-run exec-status fast-fail change integrates cleanly with #512/#513
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\subagents\workflows\wf_ae339437-da9
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\workflows\scripts\verify-cloud-run-fastfail-wf_ae339437-da9.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\workflows\scripts\verify-cloud-run-fastfail-wf_ae339437-da9.js"} to iterate without resending the script.)
Run ID: wf_ae339437-da9
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\workflows\scripts\verify-cloud-run-fastfail-wf_ae339437-da9.js", resumeFromRunId: "wf_ae339437-da9"}) ? completed agents return cached results (cached results may themselves be empty ? inspect journal.jsonl before assuming there is something to recover).

You will be notified when it completes. Use /workflows to watch live progress.

124. assistant (2026-07-07T15:48:17.967Z)

While the verification workflow runs, let me update the memory to reflect the rebase (the branch is now current on master, re-gated).

125. user (2026-07-07T15:48:24.766Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is current in your context ? no need to Read it back)

126. user (2026-07-07T15:48:31.685Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

127. assistant (2026-07-07T15:48:57.688Z)

Here's the update ? there's meaningful news since my last message.

The change landed clean, but master moved under me ? handled

When you asked, I re-checked the repo and found **`origin/master` had advanced past the base I branched from**. Two sibling PRs merged while I was working, both touching my files:

- **#513** (`23c1868`) ? added `ComputeJobSpec.skip_validation` + the `--skip-validation` worker flag (the very thing my fix is the safety net for)
- **#512** (`20862d7`, now the tip) ? the validators shared-decode refactor

So I:
1. **Rebased** `fix/cloud-run-exec-status-fastfail` forward onto the current tip (`20862d7`). My

commit is now `145854e`, parent = master's head.
2. **Verified the integration** ? #513's `skip_validation` plumbing and my fast-fail plumbing both survive intact in all overlapping files; they're orthogonal and compose. Notably, my fix is the **durable mitigation for #513's deploy-ordering hazard**: a post-#513 control-plane dispatching `--skip-validation` to an *old* worker image makes it argparse-abort *before* the marker ? exactly the ~65-min hang my change now collapses to a fast rc-5 failure.
3. **Re-ran the full gate on the final tree**: **1833 passed / 1 skipped**, coverage **85.66%** (floor 70), ruff + mypy clean.

One scare worth flagging honestly: a `git diff origin/master..HEAD` (two-dot) briefly looked like my commit was *deleting* the #512 validator files ? the "silently absorb a sibling's work" hazard. It was a **two-dot-diff artifact** from being one commit behind; the three-dot diff (`origin/master...HEAD`) confirmed my branch only ever touched its 5 files, and rebasing forward cleared it. The validation work is intact in my branch.

In progress

I've launched a background **adversarial verification workflow** (4 independent review lenses ? integration correctness, fast-fail control flow, the real Executions-API classification, and test adequacy ? each finding then adversarially refuted against the actual code). I'll report whatever survives verification when it lands.

Still not pushed / no PR (you haven't asked). The branch is current on master and merge-ready. Want me to push it and open the PR once the verification pass comes back clean?

128. user (2026-07-07T15:49:39.290Z)

pls open PRs

129. assistant (2026-07-07T15:50:03.156Z)

I'll open the PR for the change. Let me first confirm `gh` is authenticated and check whether the verification workflow has finished (so I can fold its result into the PR), then push.

130. user (2026-07-07T15:50:11.233Z)

```
=== gh available + authed? ===
gh version 2.92.0 (2026-04-28)
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
=== branch state ===
fix/cloud-run-exec-status-fastfail
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
=== any existing PR for this branch? ===
=== workflow journal (verification progress) ===
total 856
drwxr-xr-x 1 avidu 197609      0 Jul  7 21:18 .
drwxr-xr-x 1 avidu 197609      0 Jul  7 21:18 ..
-rw-r--r-- 2 avidu 197609 216573 Jul  7 21:20 agent-a720b660798f2cfad.jsonl
-rw-r--r-- 2 avidu 197609     48 Jul  7 21:18 agent-a720b660798f2cfad.meta.json
-rw-r--r-- 2 avidu 197609 204319 Jul  7 21:19 agent-a9aaabf6f747c1caf.jsonl
-rw-r--r-- 2 avidu 197609     48 Jul  7 21:18 agent-a9aaabf6f747c1caf.meta.json
-rw-r--r-- 2 avidu 197609 193716 Jul  7 21:19 agent-ab817e3361799239f.jsonl
-rw-r--r-- 2 avidu 197609     48 Jul  7 21:18 agent-ab817e3361799239f.meta.json
-rw-r--r-- 2 avidu 197609 243478 Jul  7 21:19 agent-ae8375cd5cc4e3892.jsonl
```

131. assistant (2026-07-07T15:50:19.492Z)

`gh` is authenticated (avidullu, `repo` scope) and the branch is ready. The 4 review lenses have completed and the verify stage is running. Let me check whether the workflow has produced its

final result before I open the PR:

132. user (2026-07-07T15:50:23.199Z)

```
=== journal.jsonl present? ===
-rw-r--r-- 2 avidu 197609 500 Jul  7 21:18 C:/Users/avidu/.claude/projects/C--Users-avidu-
Projects-badminton-highlight-
indexer/e83b09d8-e113-4d4d-9411-cd80ae217af9/subagents/workflows/wf_ae339437-da9/journal.jsonl
=== all agent files (count = review+verify agents) ===
4
=== tail journal (final result if done) ===
{"type":"started","key":"v2:422e5994afd64f3f17fd5b29891dd239e80a5adaa28c1cc9305f66a8c79423c6","a
gentId":"a9aaabf6f747c1caf"}
{"type":"started","key":"v2:f6d1279cd4a55071f33f85e9eb5050b7916110404744ed49468a054c14565857","a
gentId":"ae8375cd5cc4e3892"}
{"type":"started","key":"v2:e4f96397773c4c51b10d1071089664bc0be53142d6e3aef28044c650db511f3f","a
gentId":"ab817e3361799239f"}
{"type":"started","key":"v2:5b4ca917a32db79168b44c9131102dbbbcbe87918cda8eebc724a0f887a96d2a","a
gentId":"a720b660798f2cfad"}
```

133. assistant (2026-07-07T15:50:46.449Z)

The reviews are still wrapping up, but the gate is green and you've asked directly ? I'll open the PR now and fold in any confirmed verification findings as follow-up if they land. Pushing the branch:

134. user (2026-07-07T15:50:51.198Z)

```
=== pre-push re-verify: only my commit vs origin/master ===
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
=== push ===
remote:
remote: Create a pull request for 'fix/cloud-run-exec-status-fastfail' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/cloud-run-
exec-status-fastfail
remote:
branch 'fix/cloud-run-exec-status-fastfail' set up to track 'origin/fix/cloud-run-exec-status-
fastfail'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/cloud-run-exec-status-fastfail -> fix/cloud-run-exec-status-fastfail
```

135. assistant (2026-07-07T15:50:54.302Z)

Branch pushed. Now creating the PR:

136. user (2026-07-07T15:51:14.073Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/515
```

137. assistant (2026-07-07T15:51:27.806Z)

PR opened: [#515](https://github.com/Khelsutra/badminton-highlight-indexer/pull/515) ? `fix(cloud-run): fail fast when a worker execution terminates without a done-marker`.

That's the single PR for this change (one logical fix on one branch). Let me record the PR number in memory and check on the verification workflow:

138. user (2026-07-07T15:51:34.818Z)

```
=== agent files now (>4 means verify agents spawned) ===
4
=== journal events ===
{"type":"started","key":"v2:422e5994afd64f3f17fd5b29891dd239e80a5adaa28c1cc9305f66a8c79423c6","a
gentId":"a9aaabf6f747c1caf"}
```

{"type":"started","key":"v2:f6d1279cd4a55071f33f85e9eb5050b7916110404744ed49468a054c14565857","agentId":"ae8375cd5cc4e3892"}
{"type":"started","key":"v2:e4f96397773c4c51b10d1071089664bc0be53142d6e3aef28044c650db511f3f","agentId":"ab817e3361799239f"}
{"type":"started","key":"v2:5b4ca917a32db79168b44c9131102dbbbcbe87918cda8eebc724a0f887a96d2a","agentId":"a720b660798f2cfad"}
=== PR #515 status ===
{"additions":375,"changedFiles":5,"deletions":7,"mergeable":"MERGEABLE","number":515,"state":"OPEN","title":"fix(cloud-run): fail fast when a worker execution terminates without a done-marker","url":"https://github.com/Khelsutra/badminton-highlight-indexer/pull/515"}

139. assistant (2026-07-07T15:51:43.811Z)

PR #515 is **OPEN, MERGEABLE**, +375/?7 across exactly the 5 files. The verification workflow's review agents are still running (thorough multi-file reads) ? I'll report anything that survives verification when it completes. Let me record the PR in memory:

140. user (2026-07-07T15:51:46.361Z)

<tool_use_error>String to replace not found in file.
String: description: "Local UNPUSHED branch fix/cloud-run-exec-status-fastfail ? run_infer now polls the Executions API to fail fast when a worker crashes before the done-marker"
(note: Edit also tried swapping \uXXXX escapes and their characters; neither form matched, so the mismatch is likely elsewhere in old_string. Re-read the file and copy the exact surrounding text.)</tool_use_error>

141. user (2026-07-07T15:52:01.936Z)

```
1       ---
2       name: cloud-run-exec-status-fastfail
3       description: Local UNPUSHED branch fix/cloud-run-exec-status-fastfail ? run_infer now
polls the Executions API to fail fast when a worker crashes before the done-marker
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: e83b09d8-e113-4d4d-9411-cd80ae217af9
8       ---
9
10      **Branch `fix/cloud-run-exec-status-fastfail`** (commit `145854e`, **rebased onto
current
11      `origin/master` @ 20862d7** = master with #513 skip-validation + #512 validator-decode
both merged)
12      ? **local, gated, NOT pushed** (like [[compute-local-hosted-guardrail]] / [[validation-
shared-decode-pass]]).
13      Full gate reproduced on the final rebased tree: full suite **1833 passed /1 skipped**,
coverage
14      **85.66%** (floor 70), `ruff` + `mypy backend training` clean. Only my 5 files change
(verified via
15      `git diff origin/master...HEAD` ? three-dot; a two-dot diff briefly looked like it
"deleted" the #512
16      validator files, but that was just being one commit behind, since fixed by rebasing
forward).
17
18      **What it fixes:** `CloudRunCompute.run_infer` (backend/compute/cloud_run.py) bucket-
polled the
19      done-marker ONLY; a worker that terminated before writing the marker (old image
argparse-aborting
20      on an unknown flag like `--skip-validation`, OOM, any pre-marker exception) made the
dispatcher
21      poll the full `deployment.remote_poll_max_wait_sec` (~65 min) then fail opaquely.
22
23      **How:** each poll iteration (`_poll_check`) checks the marker FIRST (sole success
signal ? a
24      worker SUCCESS *or FAILURE* marker still wins), and when it's absent probes the Cloud
```

```
Run
25      Executions API via the new NON-abstract `CloudRunJobClient.execution_terminal_state`
(default
26      `None` = additive/default-OFF; real impl uses `get_execution` completion_time + per-task
counts).
27      A terminal execution + no marker ? `RemoteExecutionFailed` (new, in
`backend/compute/base.py`)
28      after ONE final marker re-check to close the write race. `_dispatch_remote` maps it to
**rc 5**
29      (distinct from rc 4 poll-timeout, which means the exec may still be running). Probe is
best-effort:
30      a raising/flaky status API is swallowed ? keep polling, never fails a healthy run.
31
32      **Why it exists:** surfaced by the adversarial review of the worker-skip-revalidation PR
33      (`fix/worker-skip-revalidation-on-dispatch`), whose interim mitigation was a "deploy
control-plane
34      + worker together" note. This is the durable fix for the whole "worker crashed before
the
35      done-marker" class.
36
37      **Next:** owner to push + open PR (I did not push/PR ? not asked). Real-client
`execution_terminal_state`
38      SDK body is `# pragma: no cover` (never run in CI, same convention as
`run_job`/`cancel_execution`).
39
```

142. user (2026-07-07T15:52:06.861Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

143. user (2026-07-07T15:52:13.699Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

144. user (2026-07-07T15:52:19.413Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

145. user (2026-07-07T15:52:27.236Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

146. assistant (2026-07-07T15:52:38.853Z)

```
Done ? PR is open.
```

[PR #515](https://github.com/Khelsutra/badminton-highlight-indexer/pull/515) ? `fix(cloud-run): fail fast when

```
- **State:** OPEN, **MERGEABLE**, +375/?7 across the 5 files
- Based on current `master` (`20862d7`, which already has #512 + #513); `git diff master...HEAD`
touches only my files
- Body documents the problem, the fix, the integration with #513 (this is the durable
replacement for #513's "deploy-together" interim note), and the local gate results (**1833
passed / 1 skipped**, coverage 85.66%, ruff + mypy clean)

This is the **one** PR for this change ? a single logical fix on one branch. If by "PRs" you had
```

another change in mind (the other in-flight branches like the compute=local guardrail or validation-decode are already merged per the memory index), tell me which and I'll open it.

Still running

The adversarial verification workflow (4 review lenses + refute pass) is finishing up. I'll report any finding that survives verification, and fold a fix into the branch **before** merge if something real turns up ? the PR will update automatically. I have **not** merged it (that's your call, and I'd want the verification pass back first).

147. user (2026-07-07T15:54:09.534Z)

<task-notification>
<task-id>wah0pctzi</task-id>
<tool-use-id>toolu_01XL5cDD6sPPKaRKPxhsZ5k8</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\tasks\wah0pctzi.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Adversarially verify the rebased cloud-run exec-status fast-fail change integrates cleanly with #512/#513" completed</summary>
<result>{"surviving":[{"dimension":"realclient","severity":"blocker","category":"correctness","file":"backend/compute/cloud_run.py","line":190,"summary":"GoogleCloudRunJobClient.execution_terminal_state ALWAYS returns None on the real client, so the fast-fail path is completely inert in production and the ~65-min hang this commit exists to collapse is not actually fixed.","detail":"The terminal-detection guard reads `completion_time.seconds`:\n\n   if not getattr(getattr(exe, \"completion_time\", None), \"seconds\", 0):\n   return None\n\nThe comment asserts \"completion_time is a proto Timestamp\", but google-cloud-run (run_v2) is a proto-plus library, and proto-plus marshals `google.protobuf.Timestamp` fields to native Python datetimes, NOT to a proto Timestamp with a `.seconds` field. I verified this against the installed `google-cloud-run` / proto-plus 1.28.0:\n - UNSET completion_time (still running) -> exe.completion_time is None\n - SET   completion_time (terminal)   -> exe.completion_time is a proto.datetime_helpers.DatetimeWithNanoseconds (a datetime.datetime subclass)\nA datetime has `.second` (singular), never `.seconds`. So `getattr(&lt;datetime&gt;, \"seconds\", 0)` is always 0, `not 0` is always True, and the function returns None on line 190 for EVERY execution -- including a definitively-terminal, completion_time-set, failed_count=1 one. I reproduced the real classifier body end-to-end: classify(terminal-Failed execution) => None (expected 'Failed'). The cancelled/failed/succeeded classification below (lines 193-199) is therefore unreachable dead code on the real path.\n\nEvery unit test passes only because the fakes (_CrashedNoMarkerClient, _MarkerAndFailedStatus, etc.) OVERRIDE execution_terminal_state to return a plain string, so they never exercise the real proto-parsing method (which is pragma:no-cover). The seam looks wired but the sole real implementation never signals a terminal state.\n\nNet effect: on the only path that matters (production Cloud Run), _terminal_execution_state always sees None, _poll_check always returns None, and run_infer degrades to exactly the pre-commit marker-only polling -- i.e. the fix is a no-op in prod while giving green-test false confidence.\n\nFix: distinguish running vs terminal via the marshaled datetime itself, e.g. `if getattr(exe, \"completion_time\", None) is None: return None` (proto-plus returns None for an unset Timestamp and a real datetime once terminal). Be defensive about a possible epoch-datetime for unset in other proto-plus versions.","failure_scenario":"The exact motivating scenario: post-#513 control-plane dispatches `--skip-validation` to an OLD worker image with no such flag. The worker argparse-aborts (exit 2) BEFORE writing any done-marker. The Cloud Run execution reaches terminal Failed (completion_time set, failed_count=1). On the next bucket-poll, _poll_check -> _terminal_execution_state -> real execution_terminal_state calls get_execution, but the completion_time.seconds guard evaluates `not getattr(&lt;DatetimeWithNanoseconds&gt;, 'seconds', 0)` == `not 0` == True and returns None. _poll_check returns None and keeps polling. RemoteExecutionFailed/rc-5 NEVER fires; the run hangs the full remote_poll_max_wait_sec (~65 min) then raises PolicyTimeout (rc 4) -- precisely the opaque long hang this commit claims to eliminate.","verdict":"CONFIRMED","verify_reasoning":"Reproduced against the installed google-cloud-run + proto-plus 1.28.0. run_v2.Execution is a proto-plus message; its completion_time (a google.protobuf.Timestamp field) marshals to None when unset and to a proto.datetime_helpers.DatetimeWithNanoseconds (a datetime.datetime subclass) when set. That subclass exposes .second (singular) and has NO .seconds attribute (hasattr==False). Therefore the terminal guard at cloud_run.py:190 ? `not getattr(getattr(exe, \"completion_time\", None), \"seconds\", 0)` ? evaluates to `not 0 == True` for EVERY execution: unset

(getattr(None,\"seconds\",0)==0) and terminal alike
(getattr(<datetime>,\"seconds\",0)==0). I ran the exact classifier body (lines 189-199) on
real proto-plus Execution messages: classify(terminal Failed, failed_count=1) => None
(expected \"Failed\"); Cancelled => None; Succeeded => None. So execution_terminal_state
always returns None on the real client and lines 193-199 are unreachable dead code. Consequently
_terminal_execution_state -> _poll_check always returns None and run_infer degrades to
exactly the pre-commit marker-only polling; the motivating scenario (old worker argparse-aborts
before writing a marker, execution terminal Failed) never fires RemoteExecutionFailed/rc-5 and
still hangs the full remote_poll_max_wait_sec (~65 min) into a PolicyTimeout/rc-4. Tests stay
green only because every fake overrides execution_terminal_state to return a string and the real
proto-parsing body is pragma:no-cover (the one real-client test only hits the unresolvable-name
early return). The commit's sole production purpose is defeated while giving false green-test
confidence. The proposed fix (`if getattr(exe, \"completion_time\", None) is None: return None`)
is validated by the same reproduction. This is a real, production-path correctness defect, not s
tyle/speculation.","corrected_severity":"blocker"}],"refuted_summaries":[],"assessments":[{"lens
":"integration","note":null},{"lens":"faillogic","note":null},{"lens":"realclient","note":null},
{"lens":"tests","note":null}]}</result>
<diagnostics>Per-agent results: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
badminton-highlight-
indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\subagents\workflows\wf_ae339437-da9/journal.jsonl ?
one {"type":"result",...} line per completed agent with its full return value.
If the result above is empty or unexpected, Read this file BEFORE diagnosing ? do not assume
agents returned non-empty results.
To re-run with edited post-processing: Workflow({scriptPath: 'C:\Users\avidu\.claude\projects\C
--Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\workflows\scripts\verify-cloud-run-fastfail-
wf_ae339437-da9.js', resumeFromRunId: 'wf_ae339437-da9'}) ? agents whose (prompt, opts) are
unchanged replay from cache.</diagnostics>
<usage><agent_count>5</agent_count><agents_done>5</agents_done><agents_error>0</agents_error><ag
ents_skipped>0</agents_skipped><agents_empty_result>0</agents_empty_result><subagent_tokens>4661
02</subagent_tokens><tool_uses>67</tool_uses><duration_ms>356386</duration_ms></usage>
</task-notification>

148. assistant (2026-07-07T15:55:14.322Z)

The verification caught a **genuine blocker** ? and it's serious. My original
`execution_terminal_state` used `completion_time.seconds`, but `google-cloud-run` is a **proto-
plus** library that marshals `Timestamp` fields to Python `datetime` objects, which have
`.second` (singular), not `.seconds`. So `getattr(dt, "seconds", 0)` is always `0` ? the guard
always returns `None` ? the whole fast-fail path would be a **silent no-op in production**,
passing tests only because the fakes override the method. The tests are green for the wrong
reason.

This is exactly why the real-client body being `# pragma: no cover` was a risk. Let me first
confirm the proto-plus behavior empirically (is the SDK installed here?), then fix it ? and
critically, restructure so the classification logic is **testable** instead of hidden behind the
pragma.

149. user (2026-07-07T15:55:26.129Z)

```
=== is google-cloud-run / proto-plus installed here? ===
run_v2 OK; proto-plus 1.28.0
=== empirical: unset vs set completion_time under proto-plus ===
UNSET completion_time repr: None | type: NoneType | is None: True
UNSET has .seconds?: False | has .second?: n/a
SET completion_time repr: DatetimeWithNanoseconds(2026, 7, 7, 12, 0,
tzinfo=datetime.timezone.utc) | type: DatetimeWithNanoseconds
SET has .seconds?: False | has .second?: True
getattr(SET,'seconds',0) = 0  <-- the bug: always 0
default failed_count: 0 int
set failed_count: 1 | getattr works: 1
Condition.State values: ['STATE_UNSPECIFIED', 'CONDITION_PENDING', 'CONDITION_RECONCILING',
'CONDITION_FAILED', 'CONDITION_SUCCEEDED']
Condition has type_?: True | has type?: True
```

150. assistant (2026-07-07T15:56:20.398Z)

Empirically **confirmed** ? this is a real blocker, and my original code would have been a silent no-op in production:
- unset `completion_time` ? `None`; set ? a `DatetimeWithNanoseconds` (datetime subclass) with `.second`, **not** `.seconds`
- `getattr(set_datetime, "seconds", 0) == 0` ? my guard returned `None` for *every* execution

The correct terminal test is `completion_time is None`. And I'll fix the deeper problem: **extract the classification into a pure, testable function** so this can't hide behind `# pragma: no cover` again. Let me read the current method region first (line numbers shifted after rebase):

151. user (2026-07-07T15:56:21.711Z)

```
168                 run_v2.ExecutionsClient()
169             )  # pragma: no cover - exercised only on a real run
170             client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172         def execution_terminal_state(self, execution: str) -> Optional[str]:
173             # Advisory status probe (see the ABC docstring): describe the execution and,
ONLY once it
174             # is terminal (``completion_time`` set), classify it. A non-resolvable name /
absent SDK
175             # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this
never
176             # raises on bad input, because the probe is best-effort and must not fail a
healthy run.
177             if "/executions/" not in (execution or ""):
178                 return None
179             try:
180                 from google.cloud import run_v2  # type: ignore[attr-defined]
181             except Exception:  # pragma: no cover - real SDK not present in CI
182                 return None
183             client = (
184                 run_v2.ExecutionsClient()
185             )  # pragma: no cover - exercised only on a real run
186             exe = client.get_execution(  # pragma: no cover - real run only
187                 request=run_v2.GetExecutionRequest(name=execution)
188             )
189             # completion_time is a proto Timestamp: unset ? seconds==0 ? still
running/pending.
190             if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
191                 return None  # pragma: no cover - real run only
192             # Terminal: prefer the explicit per-task counts (a cancel shows as
cancelled_count>0).
193             if getattr(exe, "cancelled_count", 0):  # pragma: no cover - real run only
194                 return "Cancelled"
195             if getattr(exe, "failed_count", 0):  # pragma: no cover - real run only
196                 return "Failed"
197             if getattr(exe, "succeeded_count", 0):  # pragma: no cover - real run only
198                 return "Succeeded"
199             return "Failed"  # pragma: no cover - terminal but no positive success ? treat
as failed
200
201
202     class CloudRunCompute(ComputeBackend):
203         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
```

152. assistant (2026-07-07T15:56:25.979Z)

Now the fix ? replace the broken method body with one that delegates to a pure, testable classifier, and add the classifier as a module-level function:

153. user (2026-07-07T15:56:40.760Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

154. assistant (2026-07-07T15:56:44.705Z)

Now add the two regression tests ? one CI-runnable (fakes mimicking proto-plus shape), one that exercises the **real** `run_v2.Execution` (skips where the SDK is absent). This is the coverage that would have caught the bug:

155. user (2026-07-07T15:56:58.905Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_compute_backend.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

156. assistant (2026-07-07T15:57:04.641Z)

Now run the tests ? including the real proto-plus test (which executes here since the SDK is installed) ? and confirm the classifier is covered:

157. user (2026-07-07T15:57:26.394Z)

```
...........                                                              [100%]
11 passed, 22 deselected in 3.14s
=== coverage of the classifier (must be covered now, not pragma-hidden) ===
backend\compute\cloud_run.py     140     19    86%   124-143, 160-170, 179-180, 301-306,
389-394, 444-446
TOTAL                            140     19    86%
33 passed in 10.67s
```

158. assistant (2026-07-07T15:57:49.333Z)

33 passed ? the real proto-plus test ran here (SDK installed) and confirms the fix works against **actual** `run_v2.Execution` objects, and `_classify_execution_state` is now covered (no longer pragma-hidden). Let me run the full gate before updating the PR:

159. user (2026-07-07T16:02:29.108Z)

```
=== ruff ===
All checks passed!
=== mypy ===
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 149 source files
=== FULL SUITE + COVERAGE (floor 70) ===
42 files skipped due to complete coverage.
Required test coverage of 70% reached. Total coverage: 85.68%
1835 passed, 1 skipped, 25 warnings in 268.78s (0:04:28)
================================================================================
 RESULT: PASS - all tests green
================================================================================
```

160. assistant (2026-07-07T16:02:35.072Z)

Full gate green after the fix: **1835 passed / 1 skipped**, coverage **85.68%**, ruff + mypy clean. Committing the blocker fix as a new commit (squash-merge will collapse it) and pushing to update PR #515:

161. user (2026-07-07T16:02:47.444Z)

```
=== re-verify branch + staged paths ===
fix/cloud-run-exec-status-fastfail
 backend/compute/cloud_run.py  | 42 ++++++++++++++++++----------
 tests/test_compute_backend.py | 67 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 94 insertions(+), 15 deletions(-)
[fix/cloud-run-exec-status-fastfail ab03af1] fix(cloud-run): correct proto-plus Timestamp
handling in execution_terminal_state
 2 files changed, 94 insertions(+), 15 deletions(-)
=== push to update PR #515 ===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   145854e..ab03af1  fix/cloud-run-exec-status-fastfail -> fix/cloud-run-exec-status-fastfail
=== branch commits vs master ===
ab03af1 fix(cloud-run): correct proto-plus Timestamp handling in execution_terminal_state
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
```

162. assistant (2026-07-07T16:02:55.636Z)

Pushed ? PR #515 now has the fix. The other three verification lenses (integration, fail-logic, tests) surfaced nothing that survived. Let me document the finding on the PR for reviewers and update memory:

163. user (2026-07-07T16:03:06.653Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/515#issuecomment-4905802958

164. user (2026-07-07T16:03:19.489Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is current in your context ? no need to Read it back)

165. assistant (2026-07-07T16:03:40.955Z)

Done ? and the verification pass earned its keep. Here's the update.

The adversarial review caught a real blocker ? now fixed

The 4-lens verification of PR #515 surfaced one **confirmed blocker** (the other three lenses came back clean): my real-client `execution_terminal_state` would have been a **silent no-op in production**.

The bug: I gated terminality on `completion_time.seconds`. But `google-cloud-run` is a **proto-plus** library ? it marshals a `Timestamp` to a native Python `datetime` (which has `.second`, *not* `.seconds`) when set, and to `None` when unset. So `getattr(completion_time, "seconds", 0)` was **always `0`**, the method returned `None` for every execution, and the fast-fail path never fired. The full green test suite completely masked it, because every test fake overrides the method and the real body was `# pragma: no cover`. I **confirmed it empirically** against the installed proto-plus 1.28.0 before trusting the finding.

The fix (commit `ab03af1`, pushed to PR #515):
- Terminality is now `completion_time is None` (verified: unset ? `None`, set ? `DatetimeWithNanoseconds`).
- Extracted the classification into a pure, module-level `_classify_execution_state(exe)` so the logic is **unit-tested** instead of hidden behind the SDK pragma ? a fake-based test pins the proto-plus shape for CI, and an `importorskip` test exercises a **real** `run_v2.Execution` (runs here where the SDK is installed; the only thing that would catch a future proto-plus change). Only the `get_execution` network fetch stays untestable in CI.

Re-gated on the final tree: 1835 passed / 1 skipped, coverage 85.68%, ruff + mypy clean. I posted a [comment on the PR](https://github.com/Khelsutra/badminton-highlight-indexer/pull/515#issuecomment-4905802958) documenting the finding and fix.

Where things stand

- **[PR #515](https://github.com/Khelsutra/badminton-highlight-indexer/pull/515)** ? OPEN, 2 commits (feature + blocker fix; the repo squash-merges, so it collapses to one on master). Merge-ready pending your review.
- This is the one PR for this change. Not merged ? that's your call.

The lesson worth keeping (and I've noted it in memory): don't leave real-integration logic behind `# pragma: no cover` ? extract the pure decision and unit-test it, or a green suite will hide a dead-on-arrival code path exactly like this one.

166. user (2026-07-07T16:42:54.829Z)

pls merge

167. assistant (2026-07-07T16:43:14.174Z)

Merging is outward-facing, so let me verify the PR is genuinely ready first ? mergeability against the current master (which has moved again per the index), and CI status.

168. user (2026-07-07T16:43:21.267Z)

```
=== origin/master now ===
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
=== PR #515 state ===
{"baseRefName":"master","headRefName":"fix/cloud-run-exec-status-
fastfail","mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","number":515,"state":"OPEN"}
=== CI checks ===
Built wheel smoke (install + launch)    pass    1m7s    https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28880482957/job/85667051481
Dependency license scan (no copyleft in runtime deps)   pass    38s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28880482957/job/85667051528
Full test suite (offline, mocked)       pass    2m56s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28880482957/job/85667051504
Golden fixtures cross-OS (macos-latest) pass    30s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28880482957/job/85667051566
Golden fixtures cross-OS (ubuntu-latest)        pass    23s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28880482957/job/85667051521
Golden fixtures cross-OS (windows-latest)       pass    1m6s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28880482957/job/85667051758
Leak scan (PII / attribution guard)     pass    7s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28880482957/job/85667051532
Markdown link check (live docs) pass    9s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28880482957/job/85667051544
Mypy (lenient baseline) pass    33s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28880482957/job/85667051608
Ruff lint       pass    9s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28880482957/job/85667051520
Syntax sweep + import smoke (no GPU stack)      pass    22s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28880482957/job/85667051503
```

169. assistant (2026-07-07T16:43:29.433Z)

All green ? CI is **11/11 pass** (full suite, ruff, mypy, wheel smoke, license/leak scans, cross-OS golden fixtures), and the PR is **MERGEABLE / CLEAN** against the current master (which moved to #516, file-disjoint from my change). This meets the pre-merge bar. Merging with squash (repo convention) and deleting the branch:

170. user (2026-07-07T16:43:40.958Z)

```
 create mode 100644 backend/pipeline/validators/frame_sampler.py
 delete mode 100644 backend/ui/assets/index-5O3c-FCB.js
```

```
 create mode 100644 backend/ui/assets/index-BjemzRyy.css
 delete mode 100644 backend/ui/assets/index-dvIzm83F.css
 create mode 100644 backend/ui/assets/index-kJhH4dVE.js
 create mode 100644 tests/test_frame_sampler.py
=== confirm merged ===
{"mergeCommit":{"oid":"ef41473fc3c679a58f9d47c0102a7e055b9be77c"},"mergedAt":"2026-07-
07T16:43:34Z","state":"MERGED"}
=== master now includes it? ===
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
```

171. assistant (2026-07-07T16:43:50.859Z)

```
**Merged.** PR #515 squash-merged to master as
[`ef41473`](https://github.com/Khelsutra/badminton-highlight-indexer/commit/ef41473) ? it's now
the top of `origin/master`, above #516. The branch was deleted. Let me confirm the local
checkout settled cleanly and update memory:
```

172. user (2026-07-07T16:43:55.460Z)

```
=== local branch + tree ===
master
=== my change is on master (the squash commit) ===
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
3
classifier present on master ^
=== local feature branch gone? ===
deleted locally too
```

173. assistant (2026-07-07T16:44:00.476Z)

```
Confirmed: local checkout is on a clean `master`, my change (including the
`_classify_execution_state` fix) is on `origin/master` at `ef41473`, and the feature branch is
deleted both remotely and locally. Let me mark it merged in memory:
```

174. user (2026-07-07T16:44:05.149Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

175. user (2026-07-07T16:44:13.684Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

176. assistant (2026-07-07T16:44:24.035Z)

```
Merged and confirmed. ?
```

PR #515 ? merged to master

```
- **Squash-merged** as [`ef41473`](https://github.com/Khelsutra/badminton-highlight-
indexer/commit/ef41473), now the top of `origin/master` (above #516).
- **CI: 11/11 green** ? full suite, ruff, mypy, wheel smoke, dependency-license scan, leak scan,
cross-OS golden fixtures, markdown links, syntax sweep.
- Branch deleted (remote + local); local checkout is back on a clean `master` that includes the
change (classifier fix present).
```

What shipped

`CloudRunCompute.run_infer` now probes the Cloud Run Executions API as a fast negative path: a worker that terminates before writing the done-marker (crash / OOM / an old image argparse-aborting on `--skip-validation`) fails fast with **rc 5** instead of hanging the full ~65-min poll budget. The done-marker stays the sole success signal. This is the durable replacement for #513's "deploy-together" interim mitigation.

Notably, the adversarial verification pass earned its place ? it caught a proto-plus `Timestamp` bug that would have made the whole path a silent no-op in production, and that fix (plus the now-testable `_classify_execution_state`) went in before merge.

One honest caveat (recorded in memory, not blocking)

The real Executions-API `get_execution` fetch is still `# pragma: no cover` ? it can only be exercised on a live Cloud Run dispatch. The classification *logic* is now unit-tested against real proto-plus objects, but the end-to-end path is worth a one-time owner smoke (dispatch `--skip-validation` to a deliberately-old worker image ? expect a prompt rc 5) before relying on it operationally. Want me to file that as a task, or is that something you'll fold into the next real dispatch?

177. user (2026-07-07T16:45:09.096Z)

pls filw a taskj for it

178. assistant (2026-07-07T16:45:29.777Z)

I'll file it as a background task with `spawn_task` ? a self-contained prompt so it can run without this conversation's context.

179. user (2026-07-07T16:45:55.147Z)

Noted (position 1, task_id: task_069a7a19). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or superseded, call dismiss_task with this task_id. Currently pending: task_069a7a19 "Live-smoke cloud-run crash-before-marker rc-5 fast-fail". Continue your current work.

180. assistant (2026-07-07T16:46:01.186Z)

Filed as a background task chip (`task_069a7a19`) ? you can start it in a fresh worktree with one click when you're ready. Let me update the memory note to reflect that it's filed:

181. user (2026-07-07T16:46:05.993Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is current in your context ? no need to Read it back)

182. assistant (2026-07-07T16:46:14.813Z)

Done. Filed as task **`task_069a7a19`** ? "Live-smoke cloud-run crash-before-marker rc-5 fast-fail". The chip is showing; one click spins it into its own worktree session.

The task prompt is self-contained ? it explains why it exists (the `get_execution` fetch is the one `# pragma: no cover` gap), how to manufacture the exact crash-before-marker condition (pin an old worker image, dispatch `--skip-validation`), the four pass criteria (fast rc 5, not a 65-min rc 4; diagnostic names the execution; and a healthy dispatch still returns rc 0 so the probe didn't introduce false positives), plus cleanup, the region gotcha (asia-southeast1), and the branching-safety + pre-merge gate reminders.

To recap where this thread landed:
- **PR #515 merged** to master (`ef41473`, CI 11/11 green) ? the crash-before-marker fast-fail is live in the code.

- The adversarial review caught and fixed a real proto-plus blocker before merge.
- The one remaining un-CI-testable path (the live `get_execution` call) is now tracked as `task_069a7a19` for an owner smoke on real infra.